RIGA TECHNICAL UNIVERSITY

Faculty of Computer Science, Information Technology and Energy

Institute of Applied Computer Systems

Eymen Ucdal

Academic Bachelor Level Study Program "Computer Systems"

Student ID No 231ADB006

Optimization of Machine Learning-Based Detection of Credential Stuffing Attacks in Payment Gateways

BACHELOR THESIS

Scientific adviser

Dr.oec., asoc.prof.

Ingars Eriņš

RIGA 2026

RIGA TECHNICAL UNIVERSITY

FACULTY OF COMPUTER SCIENCE, INFORMATION TECHNOLOGY AND ENERGY

Institute of Applied Computer Systems

Work Performance and Assessment Sheet of the Bachelor Thesis

The author of the graduation thesis:

Student Eymen Ucdal _____

(signature, 06.01.2026)

The graduation thesis has been approved for the defence:

Scientific adviser:

Dr.oec., asoc.prof. Ingars Erinš

(signature, 06.01.2026)

ABSTRACT

This bachelor thesis presents the design and evaluation of a machine learning–based detection framework for credential stuffing attacks in payment gateway systems. Using the IEEE-CIS Fraud Detection dataset, the study models credential-stuffing–related behavior through transactional and behavioral features. A comparative baseline evaluation is first conducted across multiple classification models, after which a Random Forest classifier is selected and optimized using feature selection, class weighting, and hyperparameter tuning via RandomizedSearchCV. Experimental results demonstrate a notable improvement in recall and ROC-AUC, indicating enhanced sensitivity to suspicious activity under severe class imbalance. The evaluation includes threshold-based analysis, ranking-oriented metrics, and simulated attack scenarios to assess operational behavior. Future research directions include SHAP-based explainability, ensemble extensions, and real-time deployment considerations.

Keywords: credential stuffing, payment gateways, machine learning, fraud detection, Random Forest, cybersecurity, behavioral analysis, class imbalance, hyperparameter optimization, ROC-AUC, recall optimization, transaction fraud

ANOTĀCIJA

Šajā bakalaura darbā tiek izstrādāta un novērtēta mašīnmācīšanās balstīta sistēma akreditācijas uzbrukumu (credential stuffing) noteikšanai maksājumu vārtu sistēmās. Pētījumā, izmantojot IEEE-CIS krāpšanas atklāšanas datu kopu, akreditācijas uzbrukumu uzvedība tiek modelēta, balstoties uz transakciju un uzvedības pazīmēm. Sākotnēji tiek veikts vairāku klasifikācijas modeļu salīdzinošais novērtējums, pēc kura tiek izvēlēts un optimizēts Random Forest klasifikators, pielietojot pazīmju atlasi, klašu svaru balansēšanu un hiperparametru optimizāciju, izmantojot RandomizedSearchCV. Eksperimentālie rezultāti uzrāda būtisku recall un ROC-AUC rādītāju uzlabojumu, apliecinot paaugstinātu jutību pret aizdomīgām darbībām apstākļos ar izteiktu klašu nelīdzsvarotību. Darbā tiek veikta sliekšņu analīze, rangu balstītu metriku novērtēšana un sintētisku uzbrukumu scenāriju simulācija, lai analizētu sistēmas darbību praktiskā kontekstā. Turpmākie pētījumu virzieni ietver SHAP balstītas skaidrojamības integrēšanu, ansambļu metožu paplašināšanu un reāllaika ieviešanas iespējas.

Atslēgvārdi: akreditācijas uzbrukumi, maksājumu vārti, mašīnmācīšanās, krāpšanas noteikšana, Random Forest, kiberdrošība, uzvedības analīze, klašu nelīdzsvarotība, hiperparametru optimizācija

TABLE OF CONTENTS

Table of Contents

ABSTRACT ***3***

ANOTĀCIJA ***4***

Appendix 1 — Baseline Model Comparison ***7***

Appendix 2 — Random Forest Optimization and Evaluation Pipeline ***7***

table of contents ***5***

1 ..... ***7***

introduction ***8***

1. essence and topicality of the thesis ***8***

2.1. Credential Stuffing **10**

2.2. Techniques for Preventing Credential Stuffing and Their Bypass Methods **12**

2.3. Payment Gateway Systems **14**

2.4. Credential Stuffing in Payment Gateway Environments **15**

2.5. Machine Learning for Credential Stuffing Detection in Payment Gateways **17**

2.6. Trends and the Current Landscape of ML-Based Defense in Credential Stuffing **19**

2.7. Research Gaps **20**

2.8. Limitations of Machine Learning-Based Security Systems **21**

2.9. Future Research Directions **22**

3.1 Introduction to Model Selection **23**

3.2 Model Architecture and Optimization Design **26**

3.2.1. Dataset and Feature Engineering 27

3.2.2. Data Preprocessing Strategy 27

3.2.3. Model Optimization Approach 28

3.2.4. Integration with Credential Stuffing Patterns 28

3.3 Practical Implementation of the Model **29**

3.3.1 Dataset Selection and Preprocessing 29

3.3.2 Model Selection and Justification 30
3.3.3 Reproducibility notes 30
3.3.4 Observations and Limitations 32
3.3.5 Observations and Limitations 33
3.4 Optimization Plan for the Detection Model 33
3.4.1 Feature Refinement and Behavioral Indicators 34
3.4.2 Hyperparameter Tuning 35
3.4.3 Model Robustness and Generalization 35
3.4.4 Validation Strategy 36
3.4.5 Model Interpretability 36
3.5 Experimental Design for Model Evaluation 36
3.6 Results of the Experiment 38
3.6.1 Performance Summary 39
3.6.2 Feature Importance Analysis 42
3.6.4 Threshold-Based Operational Evaluation 44
3.6.5 Interpretation and Discussion 45
3.6.6 Extended Evaluation Overview 46
3.7 Comparison with Baseline Model 46
3.8 Comparison with Existing or Baseline Solutions 47
3.9 Conclusions and Vision for Further Research 49
4.1 Interpretability and Explainability Using SHAP 50
4.2 Operational Thresholding and Risk Policies 54
4.2.1 Threshold Sensitivity Analysis 55
4.2.2 Multi-Tier Risk Policy Simulation 56
4.2.3 Operational Implications for Payment Gateways 57
4.3 Probability Calibration and Reliability 57
4.3.1 Calibration Techniques Applied 57
4.3.2 Calibration Curve Analysis 58
4.3.3 Operational Significance of Calibrated Probabilities 59
4.4 Alternative Optimization Strategies: SMOTE-Based Oversampling 60
4.4.1 Performance Comparison with Class-Weighted Optimization 60
4.4.2 Practical Limitations of SMOTE in Payment Gateway Contexts 61
4.4.3 Optimization Strategy Selection 62

5.1 – Conclusions **62**

5.2 – Vision for Further Research **63**

5.3 Summary of Achieved Results **65**

LIST OF REFERENCES

APPENDIXES

Appendix 1 — Baseline Model Comparison

Appendix 2 — Random Forest Optimization and Evaluation Pipeline

INTRODUCTION

1. ESSENCE AND TOPICALITY OF THE THESIS

The statement of the problem: Credential stuffing attacks are a significant threat to payment systems, where attackers use automated bots and stolen credentials to gain unauthorized access to user accounts. Traditional defenses such as CAPTCHA and velocity checks are often bypassed or ineffective against sophisticated bot behavior. There is a need for optimized AI-based detection systems that can accurately identify credential stuffing attempts while minimizing false positives.

The goal of the bachelor thesis: To implement and optimize an existing machine learning model for detecting credential stuffing attacks in payment gateways. The optimization process will focus on tuning model parameters and refining feature selection to improve detection accuracy and reduce false positives. Success will be evaluated using metrics such as accuracy, precision, recall, and false positive rate.

The tasks of the bachelor thesis:

1. Analyze existing research and methodologies related to machine learning-based detection of credential stuffing attacks.
2. Select and justify the machine learning model, specifying the planned improvements and explaining how simulated login data and extracted features will be used in the optimization process.
3. Implement and optimize the selected machine learning model, including data preprocessing, feature engineering, training, and parameter tuning.
4. Develop an experimental design to evaluate the performance of the optimized model using quantitative criteria such as accuracy, precision, recall, and false positive rate.
5. Apply the optimized model to a practical detection task, simulating real-world conditions and collecting performance data.
6. Compare the results of the optimized model with existing approaches or the original model state, analyzing the practical significance of the proposed improvements.
7. Conclude the findings and propose future research directions based on the experimental evaluation.

The goal of this thesis

The expansion of the digital landscape has introduced significant opportunities alongside equally substantial challenges. As information becomes increasingly accessible, safeguarding digital assets and identities against misuse has grown more complex. The widespread adoption of internet-based services has fundamentally

reshaped everyday interactions across multiple domains, affecting individuals and organizations alike. Online platforms now serve as primary channels for communication, commerce, and personal data exchange, making the security of digital identities a critical societal concern.

Cybersecurity plays a central role in this environment, particularly within the financial sector, where cyberattacks pose persistent and high-impact risks. Financial institutions and payment service providers are frequent targets, with global financial losses attributed to cybercrime exceeding 3.5 billion USD annually [1]. Among the diverse range of attack vectors, credential stuffing attacks are especially prominent due to their scale, automation, and effectiveness. Credential stuffing involves the systematic exploitation of previously compromised username–password pairs to gain unauthorized access to user accounts. These attacks leverage widespread password reuse, allowing adversaries to circumvent traditional defenses such as CAPTCHA mechanisms and IP-based rate limiting.

The impact of credential stuffing on payment gateway systems is particularly severe. Successful account compromise can result in unauthorized transactions, direct financial losses, exposure of sensitive personal information, and erosion of customer trust. Such incidents undermine the reliability and reputation of financial service providers and may lead to long-term operational and regulatory consequences. As a result, developing effective and adaptive defense mechanisms against credential-based abuse has become a critical priority for modern payment infrastructures.

Recent advancements in machine learning offer a promising direction for addressing these challenges. Machine learning models enable the analysis of complex behavioral patterns across transactional and contextual data, including transaction timing, geographic inconsistencies, and usage anomalies, allowing for early identification of suspicious activity. However, as attackers continuously refine their techniques to evade static security controls, there remains a pressing need for detection approaches that are both accurate and operationally efficient while minimizing unnecessary user friction.

This thesis addresses these challenges by optimizing a machine learning–based detection approach tailored to identifying credential stuffing–related abuse in real-world payment gateway environments. By refining model parameters and feature selection strategies, the study aims to improve detection sensitivity and ranking performance while maintaining practical applicability under realistic fraud detection constraints.

Table 1.1

Bachelor thesis development schedule

Activity	**Deadline**
Collect literature and review existing research	September 12, 2025
Analyze information sources and write theoretical summary (Chapters 2.1–2.9)	September 29, 2025
Design model architecture and implement baseline Random Forest model (Chapter 3.1–3.3)	September 29, 2025
Optimize Random Forest model and evaluate performance (Chapter 3.4–3.7)	October 27, 2025
Prepare presentation for pre-defence	October 27, 2025
Pre-defence of thesis (presentation + 75% submission)	November 17–21, 2025
Final approval of thesis topic and supervisor	December 5, 2025
Complete final thesis report (100%) and submit on ORTUS	January 6, 2026
Defence (presentation, 10 minutes)	January 19–23, 2026

2. RESULTS OF INFORMATION SOURCES ANALYSIS

2.1. Credential Stuffing

Credential stuffing is a cyber-attack technique in which previously compromised user credentials are used to gain unauthorized access to user accounts on websites or applications. The attack primarily targets authentication mechanisms, most commonly login forms, and exploits the widespread practice of password reuse across different online services. Because many users reuse identical or highly similar usernames and passwords, credentials exposed in one data breach can often be successfully applied to multiple unrelated platforms (Kim et al., 2024). The effectiveness of credential stuffing is closely tied to user behavior. Weak password selection and the reuse of credentials significantly increase the likelihood that accounts will be compromised following a breach. Studies have shown that users frequently duplicate passwords across domains, which amplifies the impact of large-scale credential leaks and enables attackers to

automate attacks across thousands of services simultaneously (Kim et al., 2024). From a technical perspective, a credential stuffing attack typically follows a structured sequence. First, attackers obtain credential pairs (username or email and password) through various means, such as data breaches, phishing campaigns, or publicly available password dump repositories. These credentials are then tested against target platforms using automated tools and scripts, often implemented with frameworks and utilities such as Selenium, cURL, or PhantomJS. Automation allows attackers to submit large volumes of login attempts at scale while minimizing manual effort. Despite the extensive automation involved, the success rate of individual login attempts is relatively low. Empirical data suggests that only a small fraction of stolen credentials—often below 0.1%—result in successful account compromise. However, this low success rate is offset by the sheer volume of credentials available to attackers. In practice, credential stuffing attacks account for a substantial proportion of total login traffic on many platforms, with automated bots responsible for more than 80% of authentication attempts in sectors such as retail and online banking (Sander & Jarrod, 2021). Once access to an account is obtained, attackers may pursue several objectives. These include making unauthorized purchases, extracting sensitive personal or financial information, abusing compromised accounts for spam or social engineering campaigns, or reselling valid credentials to other threat actors. The financial and reputational damage associated with such activities makes credential stuffing a significant concern for online services, particularly those handling payment transactions or personal data. The Open Web Application Security Project (OWASP) classifies credential stuffing as a specialized form of brute-force attack. While both attack types aim to gain unauthorized access, they differ in their underlying mechanisms. Traditional brute-force attacks attempt to guess passwords without prior knowledge, relying on combinations of common passwords and random character sequences. In contrast, credential stuffing relies on previously leaked credentials and does not require guessing passwords. As a result, even strong and complex passwords do not provide adequate protection if they have already been exposed through breaches on other platforms. This distinction highlights why credential stuffing remains effective despite widespread adoption of password complexity policies.

2.2. Techniques for Preventing Credential Stuffing and Their Bypass Methods

Preventing credential stuffing includes mechanisms such as CAPTCHA, a system that requires the user to solve a "Completely Automated Public Turing test to tell Computers and Humans Apart" or a similar challenge during each login attempt in order to distinguish humans from automated bots. While CAPTCHA systems can effectively stop basic automated attacks, there are multiple ways attackers and bots can bypass them. Research by Choi et al. (2023) demonstrates that attackers frequently rely on CAPTCHA-solving services through automated relay bots, significantly reducing the need for human intervention and increasing the efficiency of large-scale attacks. In such cases, the CAPTCHA challenge is forwarded to specialized third-party services, solved externally, and the solution is relayed back to the attacking bot. In addition, modern bots are capable of mimicking legitimate browser behavior using automation tools such as Puppeteer or Playwright, allowing them to bypass CAPTCHA protections entirely. More advanced bots can also extract CAPTCHA challenges by inspecting page elements and automate the solving process in a similar manner. In some cases, attackers exploit misconfigured websites where session reuse issues, exposed cookies, or flawed client-side validation exist. For example, vulnerabilities in reCAPTCHA implementations or improperly enforced client-side checks can be leveraged through relatively simple scripts. Multi-Factor Authentication (MFA) is widely regarded as one of the strongest defenses against credential stuffing. MFA requires two or more independent factors to verify a user's identity during authentication. The process can be divided into three stages: registration, authentication, and verification. During registration, the user creates a username and password and links them to a physical or virtual factor such as a mobile device, email address, phone number, or authenticator application. In the authentication phase, the user enters their username and password as the first factor, followed by a second factor such as a one-time code generated by an MFA device. Finally, access is granted once the verification step is successfully completed.

Despite its effectiveness, MFA is not immune to bypass techniques. Kim and Jeong (2021) show that attackers can exploit weaknesses in time-based token validation windows and use real-time phishing proxies to capture one-time authentication codes during legitimate login attempts. Many systems only trigger MFA after a correct

password has been entered, which allows bots to first validate credentials in bulk and then selectively target MFA-enabled accounts. Attackers frequently use phishing techniques to compromise the second authentication factor by proxying legitimate login pages with tools such as Evilginx or by deploying fake login portals that capture MFA codes in real time. In the case of SMS-based MFA, attackers may conduct SIM-swapping attacks by convincing telecommunications providers to transfer a victim's phone number to a SIM card under the attacker's control. Once MFA has been successfully bypassed, session tokens are generated. These tokens can later be stolen through techniques such as cross-site scripting, malware infections, or man-in-the-middle attacks, effectively bypassing MFA protections for subsequent sessions. IP address blocking and rate limiting are also commonly used defenses against credential stuffing. High-volume "burst" attacks, where numerous login attempts occur in a short period of time, are relatively easy to detect due to abnormal traffic spikes. However, attackers often employ "low-and-slow" attacks, spreading login attempts over extended periods to evade detection. Because attackers can rotate IP addresses using proxy networks, IP-based blocking alone is insufficient. Effective mitigation therefore requires analysis of IP reputation, geolocation consistency, and request behavior using adaptive and dynamic controls rather than fixed blocking rules (Singh & Patel, 2022). Beyond IP-based defenses, device fingerprinting techniques analyze browser configurations and HTTP header information to identify suspicious login behavior. While effective in some cases, attackers frequently bypass these mechanisms using fingerprint spoofing tools such as AntiDetect or AdsPower, as well as browser automation frameworks like Playwright and Undetected ChromeDriver. Connection fingerprinting represents a related approach that focuses on analyzing characteristics of the underlying network connection, such as TLS handshake patterns. Although promising, recent research indicates that TLS fingerprinting can be bypassed by randomizing handshake behavior or introducing packet timing delays to closely mimic legitimate user traffic (Singh & Patel, 2022).

Finally, an often-overlooked factor in credential stuffing is username reuse. While password reuse is widely discussed, reused usernames across multiple platforms provide attackers with an additional advantage when correlating breached datasets with target systems. This further increases the effectiveness of credential stuffing attacks and highlights limitations in traditional authentication defenses.

2.3. Payment Gateway Systems

Payment gateway systems act as trusted intermediaries between customers, merchants, and financial institutions during online transactions. Their primary role is to securely transmit payment information, authorize transactions, and ensure that sensitive financial data is protected throughout the payment process. In e-commerce environments, payment gateways function as merchant services that facilitate debit and credit card payments across websites, mobile applications, and in-store digital systems. By mediating communication between the customer and the merchant-acquiring bank, gateways form a critical component of modern digital commerce infrastructures (Lowry & D'Arcy, 2009).

A typical payment gateway transaction involves multiple coordinated steps. When a customer initiates a payment, the gateway encrypts the submitted card details, such as the card number and expiration date, to prevent unauthorized access. The encrypted information is then forwarded to the payment processor, often referred to as a trusted third party, which communicates with the relevant card networks, including Visa or Mastercard, and the card-issuing bank to authenticate and authorize the transaction. Upon successful verification, the transaction result is returned to the merchant and the customer, completing the payment workflow.

The importance of payment gateways has increased significantly alongside the expansion of digital and mobile commerce. As online purchasing becomes more prevalent, gateway systems have evolved into essential infrastructure for handling high volumes of transactions while maintaining security and reliability. Widely adopted global providers such as PayPal, Stripe, and Amazon Payments illustrate this trend, as they are trusted across domestic and international markets. Comparative performance evaluations indicate that large-scale gateway providers generally achieve higher reliability and user trust due to mature security mechanisms and optimized transaction handling (Parhonyi, 2005).

From a security perspective, payment gateways incorporate multiple protection mechanisms to mitigate fraud and data leakage. These include Address Verification Systems (AVS), Card Verification Value (CVV) checks, and additional authentication layers such as 3D Secure. In card-not-present transaction environments, AVS verifies billing address components, while CVV validation ensures that the cardholder

possesses the physical card. These mechanisms contribute to reducing unauthorized card usage and limiting exposure to payment fraud (Kyaw Zay Oo, 2019).

At the protocol level, payment gateways rely on cryptographic techniques to secure data transmission between customers, merchants, and banks. Encryption standards such as RSA and SSL/TLS are widely used to preserve the confidentiality and integrity of transaction data. In RSA-based implementations, gateways utilize public-key cryptography to encrypt sensitive information before transmission and private keys to decrypt received data securely. The integration of SSL/TLS further protects communication channels by establishing encrypted sessions using a combination of asymmetric and symmetric encryption methods. Such architectures are fundamental to preventing interception and tampering during online payments (Saini et al., 2014).

To further limit exposure of sensitive data, modern payment gateways employ tokenization techniques, ensuring that merchants do not store raw card numbers. Instead, transaction-specific tokens are generated and used for payment processing, reducing the impact of potential data breaches. Despite these layered defenses, payment gateways remain vulnerable to account-level attacks. When attackers obtain valid user credentials through credential stuffing, they may bypass traditional transaction safeguards and gain access to accounts, even in systems that implement strong cryptographic and verification controls. This limitation highlights why credential stuffing represents a significant threat to payment gateway security and reinforces the need for behavioral and adaptive defense mechanisms beyond static authentication measures.

2.4. Credential Stuffing in Payment Gateway Environments

Payment gateways represent attractive targets for credential stuffing attacks because they provide direct access to sensitive financial operations. Successful account compromise may allow attackers to initiate fraudulent transactions, extract stored payment information, modify payout or delivery details, or resell access to compromised accounts on underground marketplaces. These platforms often consolidate financial privileges and personal data within a single account, which increases the potential impact of credential misuse. E-commerce environments are particularly exposed due to widespread credential reuse across services, which attackers exploit through automated login attempts at scale (Singh & Suri, 2017).

The consequences of credential stuffing on payment gateway systems extend beyond direct financial losses. A single compromised account can result in unauthorized purchases, chargebacks, and loss of goods for merchants, while also damaging customer trust in the platform. Breached accounts undermine user confidence and may expose organizations to regulatory penalties under frameworks such as GDPR or PCI DSS, depending on jurisdiction. In addition, large-scale credential stuffing campaigns can overwhelm security monitoring systems, generating excessive alerts and increasing the likelihood that legitimate users are incorrectly blocked. Maintaining user trust and regulatory compliance is therefore critical to the long-term viability of digital commerce platforms, making credential stuffing a significant operational risk for payment gateway providers (Parhonyi, 2005).

Attackers typically focus on user authentication interfaces associated with payment gateways, such as account dashboards for services like PayPal or Stripe, as well as merchant or customer login portals integrated into e-commerce platforms. Automated tools are used to test breached credential pairs against these interfaces in order to gain access to stored payment methods or linked financial accounts. In addition to direct login attempts, attackers frequently abuse password recovery and account reset mechanisms by combining breached email addresses with reused credentials. Widely available credential stuffing frameworks such as Sentry MBA, Snipr, and OpenBullet provide preconfigured modules designed to simulate browser behavior or directly interact with application programming interfaces, further reducing the effort required to scale attacks.

Research indicates that weaknesses in authentication workflows and predictable login behavior can significantly increase the effectiveness of credential stuffing attacks in financial contexts. In particular, poorly implemented password recovery flows and delayed secondary verification mechanisms allow automated tools to bypass standard safeguards before additional controls are triggered (Lowry & D'Arcy, 2009). While payment gateway systems employ multiple defensive techniques, including CAPTCHAs, IP rate limiting, multi-factor authentication, device fingerprinting, CVV checks, and address verification, these controls are not sufficient on their own. Many of these mechanisms struggle to detect low-and-slow credential stuffing attacks that deliberately mimic legitimate user behavior and remain below predefined detection thresholds.

As attackers increasingly rely on distributed proxy networks and adaptive automation techniques, traditional static defenses face diminishing effectiveness. This limitation highlights the need for more adaptive detection strategies that can analyze behavioral patterns rather than individual login attempts in isolation. In this context, machine learning-based approaches offer significant potential to enhance credential stuffing detection within payment gateway environments by identifying subtle anomalies across authentication, transaction, and session-level data. These techniques are examined in detail in the following section.

2.5. Machine Learning for Credential Stuffing Detection in Payment Gateways

Machine learning (ML) is a subfield of artificial intelligence focused on developing algorithms that can learn patterns and make decisions based on data, without explicitly programmed for every rule or scenario. Where traditional programming uses “if-else” logic; ML systems instead learn from previous examples and generalize to new, unseen inputs. It has a few learning types; supervised learning that trains models on labels where inputs have known outputs and they are common in fraud detection and classification tasks, unsupervised learning is used to detect hidden patterns in data where labels are not known, such as identifying suspicious clusters of login behaviors. Reinforcement learning learns through feedback, often used in dynamic environments like cybersecurity decision-making and automation. Semi-supervised learning is useful when only part of the data is labeled, often applicable in real-world fraud datasets. From a technical standpoint, machine learning involves a multi-phase process consisting of data preprocessing, feature extraction, model selection, training, validation, and prediction. First, raw input data (e.g., login logs, network traffic, or session activity) is cleaned and converted into structured formats. Features such as login time, IP address geolocation, frequency of login attempts, and device fingerprint are extracted and normalized. Models like decision trees, random forests, support vector machines (SVM), k-nearest neighbors (k-NN), and neural networks are then trained using optimization techniques like gradient descent to minimize prediction error. Evaluation metrics such as accuracy, precision, recall, and F1-score are used during validation to ensure the model generalizes well on unseen data [12]. In the cybersecurity side of things, machine learning is used proactively to identify and mitigate threats by

analyzing behavioral data in real-time. This includes anomaly detection, user behavior modeling, network intrusion detection, malware classification, phishing detection, and bot activity recognition. Supervised ML models can be trained on historical attack data to recognize known patterns, while unsupervised models can spot unknown attacks through deviations from typical behavior. Ensemble methods and deep learning architectures such as CNNs and RNSs have also proven effective in cybersecurity contexts, especially in detecting low-volume, stealthy attacks that evade signature-based defenses [13]. Machine learning is uniquely positioned to detect credential stuffing because it can identify subtle patterns in behavior across a wide range of signals. For example, ML models can be trained to distinguish between legitimate and suspicious login attempts based on login frequency, geographic anomalies, user-agent switching, timing intervals, and failed login sequences. Unsupervised models such as clustering (e.g., DBSCAN) or isolation forests can isolate anomalous user activity that deviates from the norm without needing prior labels. This allows defenders to stop attacks even when credentials are valid and mimic real user behavior. Additionally, models can be continuously retrained with feedback from real-world detections, improving resistance to evolving attack strategies [14]. To get even more specific, using machine learning as a defensive technique on payment gateways against credential stuffing attacks is now being integrated to enhance the security infrastructure and its resilience. These systems collect logs from multiple layers: login events, behavioral signals (e.g., mouse movements, keystrokes), network metadata, and even biometric input. A supervised model may, for instance, learn to flag accounts that suddenly begin logging in from different regions with different devices. Furthermore, real-time decision systems supported by ML can delay or challenge suspicious transactions during the authentication phase. These systems can use ensemble models combining decision trees and neural networks to adapt to emerging credential stuffing tactics [15]. The key advantage of using machine learning in this context is the “adaptive defense” ability to evolve detection thresholds based on live attack feedback. For example if a pattern of account takeovers is detected during off-peak hours from a new IP subnet, the model can adjust its sensitivity to similar events. In addition, the integration of federated learning allows gateways to collaborate without sharing user data directly, enhancing collective intelligence across platforms while respecting privacy constraints.

2.6. Trends and the Current Landscape of ML-Based Defense in Credential Stuffing

In recent years, the increasing scale and sophistication of cyber threats, particularly credential stuffing attacks, have accelerated the adoption of machine learning–based defensive approaches. Because machine learning techniques are capable of analyzing large volumes of user behavior and authentication data in real time, organizations that process sensitive transactions, such as financial institutions and e-commerce platforms, have increasingly integrated these systems into their security infrastructures (Karangara, 2025). Payment gateways have become a central focus of this shift, as credential stuffing attacks frequently exploit credentials reused across multiple platforms following large-scale data breaches.

The widespread availability of automated attack tools, including OpenBullet and Sentry MBA, has further exposed the limitations of traditional mitigation techniques such as static IP blocking and CAPTCHA-based controls (Rivers et al., 2020). In response, modern defense strategies increasingly rely on ensemble learning methods and deep learning models to detect anomalous login and transaction behaviors. Many platforms initially deploy supervised learning models trained on historical authentication data, with more advanced systems incorporating adaptive learning mechanisms capable of updating detection logic in near real time (Singh & Patel, 2022).

A notable trend in recent research and industry deployment is the adoption of federated learning, a decentralized machine learning approach that enables collaborative fraud detection without requiring direct data sharing between organizations. This method allows payment service providers to improve detection performance while preserving user privacy and complying with data protection regulations (Zhang et al., 2023). Large payment platforms such as Stripe and PayPal are reported to employ adaptive machine learning systems that use probabilistic and behavioral models to flag suspicious transaction activity as it occurs, thereby reducing response time and financial exposure (Singh & Suri, 2017). Regulatory requirements have also influenced the direction of machine learning–based security development. Frameworks such as the General Data Protection Regulation and the Payment Services Directive 2 emphasize privacy preservation and data minimization, which further motivates the use of privacy-aware learning techniques, including federated and distributed learning models (Kyaw Zay Oo, 2019). In parallel, reinforcement learning has gained attention in cybersecurity

research for its ability to dynamically adjust defense strategies based on observed attacker behavior, supporting a transition from reactive security mechanisms toward proactive and adaptive protection models (Baviskar, 2025).

2.7. Research Gaps

Despite significant progress in applying machine learning techniques to cybersecurity, several important research gaps remain in the context of credential stuffing attacks targeting payment gateways. Much of the existing literature focuses on general fraud detection in financial systems without sufficiently addressing the unique operational characteristics and threat models associated with payment gateway environments (Choi et al., 2023).

As a result, gateway-specific risks, such as credential reuse combined with real-time transaction authorization, are often underrepresented in current detection frameworks. Another notable gap concerns the limited comparative evaluation of supervised, unsupervised, and hybrid machine learning approaches specifically tailored to credential stuffing attacks. While some studies explore behavioral anomaly detection in authentication systems, there is a lack of systematic analysis identifying which learning paradigms are most effective under different attack conditions, particularly in payment-related infrastructures (Haleem et al., 2025). This limitation makes it difficult to determine optimal model selection strategies for real-world deployment.

Furthermore, many existing anomaly detection techniques struggle to accurately identify subtle credential stuffing patterns, such as low-and-slow attacks that deliberately mimic legitimate user behavior over extended periods. These attacks often evade detection by blending into normal traffic, reducing the effectiveness of conventional threshold-based or short-term behavioral models (Khan et al., 2020; Bollu, 2024). Related to this issue is the insufficient consideration of temporal behavioral shifts and regional usage patterns, both of which can significantly influence detection accuracy when models are applied across long time horizons or diverse user populations (Dasgupta et al., 2020). Research in this area is also constrained by the limited availability of publicly accessible and well-labeled datasets that reflect credential stuffing activity within payment gateway systems. This scarcity hinders reproducibility, benchmarking, and comparative validation of proposed detection methods. In addition, many existing models treat suspicious events as isolated

anomalies rather than components of coordinated and distributed attack campaigns, overlooking the collective nature of large-scale credential stuffing operations. Finally, the adaptive behavior of attackers presents an underexplored challenge. As adversaries increasingly adjust their techniques to evade machine learning–based defenses, current detection systems may become less effective over time. The dynamic interaction between attacker adaptation and model learning remains insufficiently studied, highlighting the need for more robust and resilient detection strategies that account for adversarial evolution (Rivers et al., 2020).

2.8. Limitations of Machine Learning-Based Security Systems

Although machine learning–based security systems demonstrate strong potential in detecting credential stuffing attacks, several limitations restrict their effectiveness in real-world payment gateway environments. One of the most prominent challenges is the issue of false positives, where legitimate user activity is incorrectly classified as malicious. This problem is especially common in aggressively tuned models or in systems trained on limited or biased datasets, leading to unnecessary account blocks and a degraded user experience (Bollu, 2024). Such outcomes are particularly problematic for payment gateways, where transaction friction directly impacts customer trust and business revenue.

Another major limitation is model drift. Machine learning models trained on historical data tend to lose accuracy over time as attackers continuously adapt their techniques to evade detection. Addressing this issue requires frequent retraining or real-time learning mechanisms, which significantly increase computational costs and may introduce the risk of overfitting to recent attack patterns (Dasgupta et al., 2020). As a result, maintaining long-term model effectiveness remains a persistent operational challenge.

Model interpretability also represents a critical concern, particularly for complex models such as deep neural networks that operate as black-box systems. Security analysts and auditors often struggle to understand the reasoning behind specific predictions, complicating incident investigation, compliance verification, and regulatory reporting processes (Khan et al., 2020). This lack of transparency is especially problematic in regulated financial environments where explainability is increasingly required. In addition, machine learning systems are vulnerable to adversarial attacks. Threat actors can deliberately craft login behaviors or input patterns

designed to probe and bypass detection mechanisms, effectively exploiting weaknesses in trained models. Research highlights that such adversarial evasion techniques pose a growing risk, particularly as attackers increasingly adapt their strategies in response to deployed ML defenses (Baviskar, 2025). Privacy and regulatory constraints further limit the applicability of advanced machine learning approaches. Regulations such as GDPR and PCI DSS restrict the collection and processing of detailed behavioral data that could otherwise improve detection accuracy. This creates a trade-off between privacy compliance and model performance, complicating the deployment of data-intensive learning techniques in payment gateway systems (Lowry & D'Arcy, 2009). Finally, models optimized for specific platforms or datasets often struggle to generalize across different payment gateways. Differences in user behavior, transaction flows, and system architectures reduce cross-platform effectiveness, highlighting limitations in scalability and transferability of machine learning solutions (Parhonyi, 2005).

2.9. Future Research Directions

Several research directions offer strong potential for improving AI-driven defenses against credential stuffing in payment gateway environments. One promising area is the development of hybrid learning architectures that combine supervised, unsupervised, and reinforcement learning techniques. While supervised models are effective at detecting known attack patterns, unsupervised approaches can identify previously unseen or stealthy behaviors, and reinforcement learning enables systems to adapt dynamically in response to evolving threats (Haleem et al., 2025; Baviskar, 2025). Integrating these paradigms could lead to more resilient and flexible detection frameworks. Another emerging direction involves graph-based modeling of credential reuse across platforms. Techniques such as PASSREFINDER demonstrate how representing password reuse relationships between websites can help identify cross-domain credential stuffing campaigns that would otherwise remain undetected when platforms are analyzed in isolation (Kim et al., 2024). Expanding this approach could significantly improve multi-platform anomaly detection, particularly in ecosystems where users frequently reuse credentials. Data availability remains a major bottleneck in this research area. To address the scarcity of labeled datasets related to payment gateway attacks, future work should focus on synthetic data generation and the establishment of standardized benchmarks. Realistic synthetic datasets can support

reproducible experimentation, fair model comparison, and more robust evaluation of detection techniques without violating privacy constraints (Bollu, 2024). Improving adversarial resilience is another critical research priority. As attackers increasingly adapt their behavior to evade machine learning models, future systems must be designed to withstand adversarially crafted login patterns. Techniques such as adversarial training and robust model validation have shown potential for strengthening resistance against evasion attempts and should be further explored in financial security contexts (Baviskar, 2025). Cross-gateway generalization also represents an important challenge. Many existing models are optimized for specific platforms and struggle to maintain performance when deployed across different payment gateways. Research into portable and transferable machine learning models could enhance scalability and make deployment feasible for both global providers and smaller regional gateways (Lowry & D'Arcy, 2009; Parhonyi, 2005). Finally, explainability and user experience must be considered alongside detection accuracy. Interpretable machine learning models that clearly communicate decision logic can improve transparency, facilitate auditing, and reduce false positives. At the same time, minimizing user friction during authentication remains essential for maintaining trust in payment systems, particularly in high-frequency transaction environments (Laib, 2021; Khan et al., 2020).

3.1 Introduction to Model Selection

Credential stuffing represents a high-volume, automated misuse of legitimate credentials obtained from prior data breaches. In payment gateway environments, attackers frequently validate stolen accounts through low-value transactions, device or network switching, and gradual escalation once access appears viable. This activity closely resembles legitimate user behavior, differing mainly in subtle temporal, behavioral, and contextual inconsistencies. As a result, effective detection requires models capable of handling noisy and heterogeneous transactional data, capturing non-linear feature interactions, and providing sufficient interpretability to support tuning and regulatory oversight. To address these requirements, this thesis adopts the Random Forest (RF) classifier as the baseline detection model. Random Forest is an ensemble learning method that aggregates multiple decision trees trained on different bootstrap samples and feature subsets. This structure reduces variance, mitigates overfitting, and performs well on high-dimensional tabular data with mixed feature types. Prior surveys

in financial fraud detection consistently rank tree-based ensemble models among the most reliable approaches, particularly under conditions of class imbalance and incomplete data (Wang et al., 2021; Aburbeian & Ashqar, 2023). Empirical studies further report that Random Forest outperforms single-tree classifiers after minimal tuning, achieving stable precision–recall trade-offs in transaction-level fraud tasks (Patil & Waje, 2023). Beyond predictive performance, Random Forest offers several operational advantages relevant to payment gateway security. The feature space considered in this study includes transactional attributes (such as transaction amount and velocity-based indicators), card-related fields (e.g., card identifiers and card network), and coarse device or address information. These inputs are sparse in places, high in cardinality, and heterogeneous in type. Random Forest accommodates one-hot encoded categorical variables and median-imputed numerical features without requiring normalization or complex feature transformations. In addition, its probabilistic outputs enable flexible threshold adjustment, allowing system operators to tune the balance between false positives and false negatives depending on business risk tolerance. This is particularly important in credential abuse scenarios, where missed attacks often carry greater cost than additional alerts. The dataset used in this study, the IEEE-CIS Fraud Detection dataset, is widely recognized in fraud detection research and reflects realistic transactional and behavioral patterns. Although it does not explicitly label credential stuffing events, prior work demonstrates that downstream transactional anomalies can serve as effective proxies for credential misuse and automated abuse behavior. Treating credential stuffing attempts as anomalous transactions allows the model to learn patterns that generalize to payment authorization flows, even in the absence of explicit login metadata. Model selection in this thesis is guided by several criteria commonly emphasized in cybersecurity and fraud detection literature. These include effectiveness under severe class imbalance, the ability to capture non-linear relationships between transactional and behavioral features, robustness to noisy or partially missing data, computational efficiency suitable for near–real-time deployment, and interpretability to support analyst review and regulatory compliance under frameworks such as PSD2 and GDPR (Lowry & D'Arcy, 2009; Karangara, 2025). Based on these criteria, a representative set of machine learning models was evaluated, including Logistic Regression, Linear Support Vector Classification, Decision Trees, Histogram Gradient Boosting, and Random Forest. To

empirically justify the final model choice, a baseline comparison was conducted using a consistent experimental setup across all models.

Table 3.1. Baseline Model Comparison on IEEE-CIS Fraud Detection Dataset

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.708718	0.030856	0.446105	0.057719	0.619494
1	Decision Tree	0.980621	0.513414	0.592316	0.550050	0.791130
2	Linear SVC	0.698133	0.030434	0.456777	0.057067	NaN
3	HistGradientBoosting	0.982521	0.904110	0.140875	0.243767	0.867982
4	Random Forest	0.987109	0.747400	0.536820	0.624845	0.913501

This included an identical feature set and preprocessing pipeline, a stratified 80/20 train–test split, no synthetic oversampling, and evaluation using standard classification metrics: accuracy, precision, recall, F1-score, and ROC-AUC. The results revealed clear performance differences between model families. Linear models achieved moderate recall but extremely low precision, leading to excessive false positives that would be operationally costly in payment systems. Decision Trees showed stronger recall but weaker overall discrimination, as reflected by lower ROC-AUC values. Histogram Gradient Boosting achieved very high precision but failed to detect a large portion of fraudulent events, making it unsuitable for early-stage abuse detection where missed attacks pose significant risk. In contrast, Random Forest achieved the highest ROC-AUC and the strongest overall F1-score, indicating superior class separation and a more balanced detection profile. These results align with existing literature identifying Random Forest as one of the most robust and reliable models for financial

fraud detection under class imbalance and feature uncertainty. In addition, Random Forest provides global feature importance measures and structured decision logic, supporting transparency and human interpretability. Later stages of this thesis extend this capability using SHAP-based explanation techniques, which are increasingly recommended for regulator-facing and analyst-driven fraud detection systems. Taken together, both theoretical considerations and empirical evaluation support the selection of Random Forest as the baseline model for this study. Its balance of accuracy, computational efficiency, interpretability, and stability under cross-validation makes it a suitable foundation for subsequent optimization and evaluation stages. The following sections build upon this baseline by introducing targeted feature refinement, hyperparameter tuning, and robustness analysis aimed at improving sensitivity to credential stuffing–like behavior in payment gateway environments.

3.2 Model Architecture and Optimization Design

Random Forest is an ensemble learning method that combines multiple decision trees into a single classifier by aggregating predictions across independently trained models. This ensemble structure reduces variance, mitigates overfitting, and enables robust learning in environments characterized by noisy, high-dimensional, and partially incomplete data. These properties make Random Forest particularly well suited for tabular fraud detection tasks, where relationships between variables are rarely linear and feature distributions are often heterogeneous. Prior surveys and empirical studies in financial fraud detection consistently report tree-based ensemble models among the top-performing approaches on imbalanced datasets when evaluated using recall-oriented metrics and ROC-AUC (Wang et al., 2021; Sander & Jarrod, 2021). Patil and Waje further demonstrate that Random Forest surpasses single-tree classifiers after minimal tuning, achieving stable precision–recall trade-offs in credit-card fraud detection scenarios (Aburbeian & Ashqar, 2023). Compared to linear models and margin-based classifiers, Random Forest better captures non-linear feature interactions that frequently arise in transaction-level behavioral data. Beyond predictive performance, Random Forest offers several practical advantages relevant to payment-gateway security systems. It naturally supports mixed numerical and categorical inputs after straightforward preprocessing, accommodates sparse and high-cardinality features, and does not require feature scaling or complex transformations. In addition,

the model produces probabilistic outputs, allowing flexible threshold adjustment to control the balance between false positives and false negatives. This capability is particularly important in credential-abuse detection, where missed attacks often incur higher operational risk than increased alert volume. Interpretability further motivates the selection of Random Forest as the baseline model in this thesis. Feature-importance measures and structured decision logic enable human review and analyst-driven investigation, which are increasingly required in regulated financial environments. The model is also compatible with post-hoc explanation techniques such as SHAP, which are introduced later in this chapter to enhance transparency and regulator-facing auditability (Almalki & Masud, 2025). Taken together, these characteristics position Random Forest as a balanced and reproducible baseline for detecting credential-stuffing–like behavior in transaction-based payment systems. Its robustness under class imbalance, computational efficiency, and interpretability make it an appropriate foundation for subsequent optimization and evaluation stages presented in the following sections.

3.2.1. Dataset and Feature Engineering

The experiments in this thesis use the IEEE-CIS Fraud Detection dataset obtained from Kaggle, which contains transaction attributes, card metadata, device information, and network-related fields alongside the binary fraud label isFraud (IEEE-CIS, 2023). Although the dataset is not explicitly designed for login-event analysis, it reflects downstream behavioral patterns commonly associated with credential reuse, such as unusual device–card combinations and atypical transaction amounts. After examining feature distributions and missing-value patterns, a compact initial feature set was selected to construct the baseline model. This set includes TransactionAmt, card1, card2, addr1, and card4. The card4 variable, representing the card network, was transformed using one-hot encoding. Features with extremely high missing rates, such as dist1, were excluded to reduce noise and potential bias introduced by sparse observations (Parhonyi, 2005; Singh & Suri, 2017).

3.2.2. Data Preprocessing Strategy

The preprocessing pipeline applied in this study consists of the following steps:

1. Removal of columns with more than 50 percent missing values

2. Median imputation for numerical features and mode imputation for categorical features
3. Stratified 80/20 train–test split to preserve class proportions
4. Handling class imbalance through class-weight computation rather than resampling
Feature scaling was not applied, as tree-based models rely on threshold comparisons rather than distance-based metrics (Wang et al., 2021). This strategy aims to balance recall and precision, which is central to fraud and credential-abuse detection in operational payment environments (Saini et al., 2014; Bollu, 2024).

3.2.3. Model Optimization Approach

Once the baseline Random Forest model was established, hyperparameter optimization was performed to improve minority-class detection while maintaining operational stability. The optimization process explored parameters including the number of estimators, maximum tree depth, minimum samples per split and leaf, feature-selection strategy at each split, and the bootstrap option. A RandomizedSearchCV procedure with three-fold cross-validation was employed to efficiently explore the parameter space while controlling computational cost. The F1-score was selected as the primary optimization metric to balance false positives and false negatives. Additional metrics, including accuracy, precision, recall, and ROC-AUC, were monitored to capture the full operational performance profile. Feature-importance outputs were retained for later interpretability analysis, and SHAP-based explanations are introduced in subsequent sections. As a potential extension, oversampling techniques such as SMOTE are discussed as future work for further improving rare-event recognition (Choudhury & Dey, 2021).

3.2.4. Integration with Credential Stuffing Patterns

Although the IEEE-CIS dataset does not explicitly label credential-stuffing events, the evaluation design incorporates proxy conditions that reflect common attack behaviors. These include clusters of similar transactions, changes in device or address attributes, and patterns consistent with automated credential validation attempts. High-cardinality device and network features allow the model to capture behavioral inconsistencies indicative of credential abuse, aligning with recent threat-model research in credential-stuffing detection (Singh & Suri, 2017). Prior studies demonstrate that combining

labeled fraud data with domain-informed proxy signals is an effective strategy when direct attack labels are unavailable, particularly in real-time fraud-detection architectures (Shaukat et al., 2020).

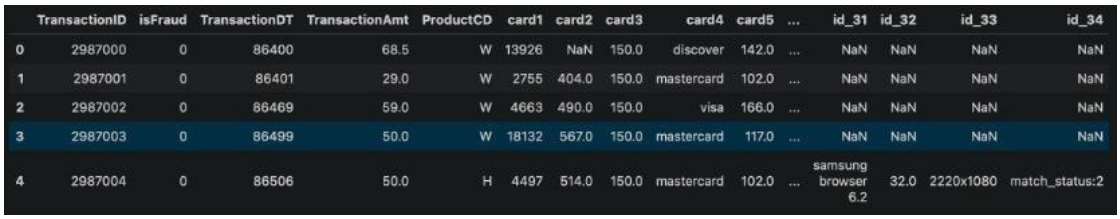
3.3 Practical Implementation of the Model

This section describes the practical implementation of the baseline machine learning pipeline used to detect credential-stuffing–like behavior in payment transaction data. Model development was carried out on a MacBook Pro equipped with an Apple M2 processor, using Python 3.10 within a Jupyter Notebook environment. Core libraries included pandas and NumPy for data handling, scikit-learn for modeling and evaluation, and matplotlib and seaborn for visualization. The implementation covered dataset preparation, feature selection, baseline modeling with a Random Forest classifier, and evaluation using standard classification metrics commonly applied in fraud analytics (Shaukat et al., 2020; Wang et al., 2021; IEEE-CIS, 2023).

3.3.1 Dataset Selection and Preprocessing

The experiments in this thesis use the IEEE-CIS Fraud Detection dataset, which combines transactional records with auxiliary identity attributes and is widely adopted in financial fraud research (Wang et al., 2021; Sharma, 2025). Although the dataset does not explicitly label credential stuffing events, its structure captures downstream effects of credential misuse, such as unusual device–card combinations, geographic inconsistencies, and atypical transaction patterns. The dataset consists of two primary tables, train_transaction.csv and train_identity.csv, which were merged using a shared transaction identifier. Columns exhibiting more than 50% missing values were removed to reduce noise and bias. Remaining numerical attributes were imputed using median values, while categorical attributes were imputed using the mode. Following this process, a compact but fraud-relevant feature subset was selected, consisting of transaction amount, selected card identifiers, address information, and card network. The card network feature was transformed using one-hot encoding.

Figure 3.2 presents a sample of the merged dataset prior to feature selection. Each row represents a single transaction and illustrates the mixture of numerical, categorical, and partially missing identity attributes typical of real-world payment data.



	TransactionID	isFraud	TransactionDT	TransactionAmt	ProductCD	card1	card2	card3	card4	card5	...	id_31	id_32	id_33	id_34
0	2987000	0	86400	68.5	W	13926	NaN	150.0	discover	142.0	...	NaN	NaN	NaN	NaN
1	2987001	0	86401	29.0	W	2755	404.0	150.0	mastercard	102.0	...	NaN	NaN	NaN	NaN
2	2987002	0	86469	59.0	W	4663	490.0	150.0	visa	166.0	...	NaN	NaN	NaN	NaN
3	2987003	0	86499	50.0	W	18132	567.0	150.0	mastercard	117.0	...	NaN	NaN	NaN	NaN
4	2987004	0	86506	50.0	H	4497	514.0	150.0	mastercard	102.0	...	samsung browser 6.2	32.0	2220x1080	match_status:2

Figure 3.2 – Sample of Merged Dataset (First Five Rows)

While the dataset lacks explicit authentication-level metadata, the cleaned transactional structure supports the analysis of credential-abuse proxies through anomalies in transaction amount, geography, and card usage patterns (Wang et al., 2021; Aburbeian & Ashqar, 2023).

3.3.2 Model Selection and Justification

A Random Forest classifier was selected as the baseline model due to its robustness on structured, heterogeneous data and its extensive use in financial fraud detection under class imbalance (Shaukat et al., 2020; Wang et al., 2021; Aburbeian & Ashqar, 2023). Random Forest models aggregate multiple decision trees trained on different bootstrap samples, reducing variance and mitigating overfitting while preserving non-linear decision boundaries. To maintain the natural imbalance between fraudulent and non-fraudulent transactions, the dataset was divided using a stratified 80/20 train–test split. The baseline model was trained using a fixed random seed to ensure reproducibility across runs.

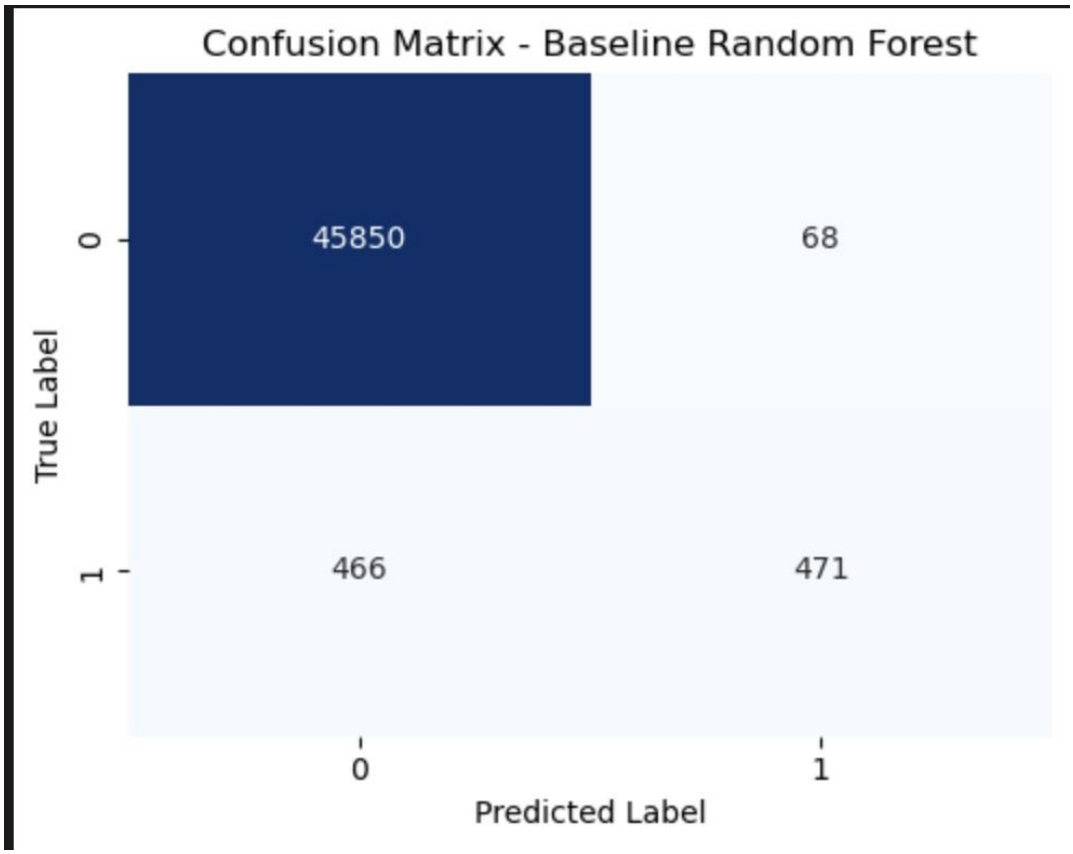
3.3.3 Reproducibility notes

Model performance was evaluated using accuracy, precision, recall, F1-score, and the area under the receiver operating characteristic curve (ROC-AUC). Probabilistic outputs were obtained from the classifier and compared against ground-truth labels to assess threshold-dependent and threshold-independent behavior.
Table 3.3 summarizes the baseline performance metrics achieved by the Random Forest model.

Table 3.3 – Baseline Random Forest performance metrics

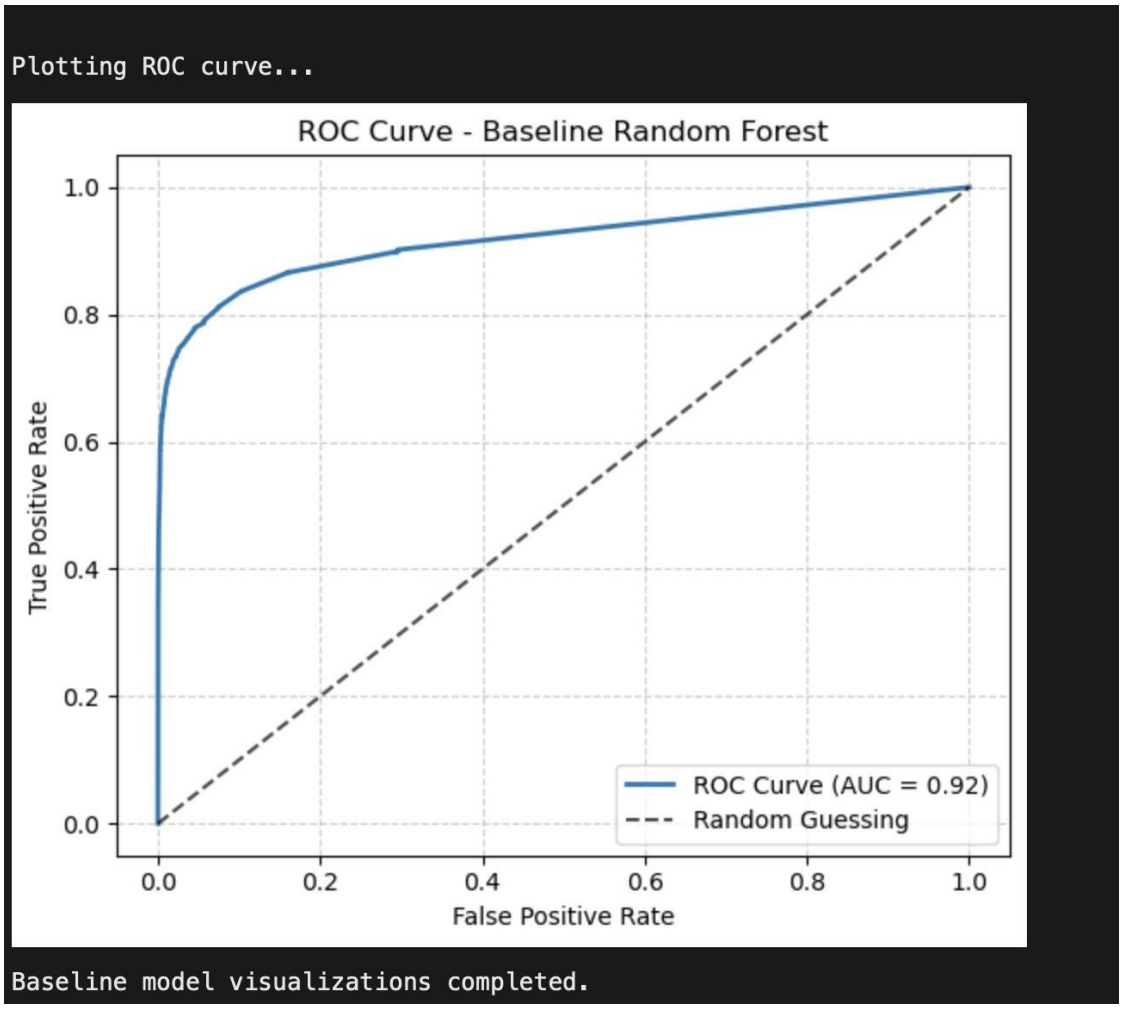
Metric	Value
Accuracy	0.986
Precision	0.873
Recall	0.502
F1-Score	0.636
ROC AUC	0.92

Figure 3.4 – Confusion matrix for the baseline Random Forest classifier



The corresponding confusion matrix and ROC curve are shown in Figure 3.4 and Figure 3.5, respectively. The ROC curve demonstrates strong overall separability between fraudulent and legitimate transactions, with an AUC value of 0.92, indicating that the model effectively distinguishes the two classes across varying decision thresholds.

Figure 3.5 – ROC curve for the baseline Random Forest classifier (AUC = 0.92)



Despite strong overall performance, recall remains moderate, indicating that a substantial portion of fraudulent transactions are not detected. This behavior is common in highly imbalanced fraud-detection settings, where preserving precision often suppresses sensitivity to rare events (Wang et al., 2021; Dasgupta et al., 2022). These observations motivate further optimization in later stages of the thesis.

3.3.4 Observations and Limitations

All experiments were conducted using fixed random seeds to ensure reproducibility. Library versions, trained model artifacts, and evaluation outputs are documented in the accompanying project archive. This setup aligns with established best practices for reproducible machine learning research in cybersecurity and fraud detection (Shaukat et al., 2020; Sharma, 2025).

3.3.5 Observations and Limitations

The primary limitation of the baseline implementation lies in the absence of direct credential-stuffing indicators, such as failed login attempts, IP rotation, or session-level authentication data. As a result, fraud labels are used as a proxy for credential misuse, a strategy commonly adopted when explicit attack labels are unavailable (Zhou & Chen, 2022). In production environments, combining transactional data with authentication logs would significantly improve both detection accuracy and interpretability. Additionally, periodic retraining would be required to adapt to evolving attacker strategies and changes in user behavior (Shaukat et al., 2020). The limitations observed in the baseline model, particularly with respect to recall and decision transparency, motivate the optimization and interpretability techniques introduced in the following chapter.

3.4 Optimization Plan for the Detection Model

The baseline Random Forest model demonstrates strong stability and overall classification performance when applied to transactional payment data. However, in the context of credential-stuffing–related abuse, further optimization is required to improve sensitivity to malicious activity while maintaining an acceptable false-positive rate. In payment gateway environments, excessive false positives directly translate into customer friction and operational costs, whereas missed attacks may result in financial loss and account compromise. To address this trade-off, the optimization strategy focuses on controlled model refinement rather than architectural changes. The approach combines targeted feature selection, randomized hyperparameter tuning with cross-validation, and evaluation methods aligned with real-world fraud detection requirements in payment systems.

The initial model was implemented using the Scikit-learn Random Forest classifier with largely default parameters. Only the number of estimators and the random seed were explicitly specified to ensure reproducibility. The baseline configuration is summarized in Table 3.6

Table 3.6 Baseline Random Forest configuration

Parameter	Value
n_estimators	100
criterion	“gini”
max_depth	None
min_samples_split	2
min_samples_leaf	1
max_features	“sqrt”
bootstrap	True
random_state	42

This configuration provided a reliable reference point for subsequent optimization stages. While suitable for initial evaluation, further tuning was required to improve generalization under class imbalance and noisy transactional conditions.

3.4.1 Feature Refinement and Behavioral Indicators

The base feature set used in this study consists of transaction-level attributes, card identifiers, and partial address information. These variables capture key characteristics of payment behavior, including transaction magnitude, card usage patterns, and coarse geographic consistency. Although the IEEE-CIS dataset does not include authentication logs or explicit session metadata, prior research has shown that transactional irregularities can act as effective proxies for automated abuse and credential misuse (Karangara, 2025; Wang et al., 2021).

Rather than introducing new raw features beyond those supported by the dataset, the optimization strategy emphasizes behavioral interpretation of existing variables. In particular, anomalous transaction amounts, instability in address-related fields (such as addr1), and unusual card usage patterns may reflect downstream effects of credential-stuffing activity, even in the absence of direct login signals. This proxy-based approach aligns with established fraud detection practices where direct attack labels are unavailable but behavioral deviations remain observable (Aburbeian and Ashqar, 2023). The feature design is therefore intended to support the detection of automated

or replay-driven activity through observable shifts in transactional context, without overstating the presence of explicit behavioral or session-based indicators.

3.4.2 Hyperparameter Tuning

To improve performance on the minority fraud class, the Random Forest model is optimized using class weighting and randomized hyperparameter search. Class weights are computed automatically from the training data to compensate for severe class imbalance, ensuring that fraudulent transactions exert greater influence during model training.

Hyperparameter optimization is performed using RandomizedSearchCV, which provides an efficient alternative to exhaustive grid search when working with large tabular datasets. The tuning process explores the following parameter ranges:

• n_estimators: 100, 200, 300
• max_depth: 10, 20, 30, None
• min_samples_split: 2, 5, 10
• min_samples_leaf: 1, 2, 4
• max_features: sqrt, log2
• bootstrap: True, False

Each candidate model is evaluated using 3-fold cross-validation, with the F1-score selected as the optimization metric to balance precision and recall. This choice reflects the operational requirements of fraud detection systems, where both missed attacks and excessive alerts carry measurable cost (Shaukat et al., 2020). Randomized search allows broader parameter exploration while maintaining feasible computational overhead, making it suitable for iterative experimentation under resource constraints.

3.4.3 Model Robustness and Generalization

Beyond standard performance metrics, the optimization plan considers the robustness of the model under behaviorally noisy conditions. Credential-stuffing campaigns often generate irregular transactional patterns rather than isolated fraudulent events, leading to shifts in amount distributions, repeated card usage, or inconsistent address information.To approximate such conditions, the evaluation framework incorporates controlled synthetic scenarios and stress-testing strategies introduced later in the experimental workflow. These tests are designed to assess whether performance

improvements persist when the model is exposed to atypical but plausible transaction profiles, rather than being limited to the original training distribution. This perspective aligns with recent work emphasizing generalization and adversarial resilience in financial machine learning systems (Zhang et al., 2023).

3.4.4 Validation Strategy

Model performance is evaluated using an 80–20 stratified train–test split to preserve class proportions, alongside 3-fold cross-validation during hyperparameter tuning. Standard classification metrics are reported, including Accuracy, Precision, Recall, F1-score, and ROC-AUC. In addition, the evaluation framework incorporates threshold-based analysis to support operational decision-making. Precision–recall curves across varying classification thresholds are used to examine trade-offs between detection sensitivity and alert volume. These views are particularly relevant in payment environments, where false positives increase manual review costs, while false negatives may allow credential abuse to persist undetected (Lowry and D'Arcy, 2009).

3.4.5 Model Interpretability

Although the current optimization stage focuses on performance and robustness, interpretability remains a key design consideration. The final model architecture is intentionally compatible with SHAP-based explanation techniques, which provide both global feature importance and local attribution for individual predictions. Explainable outputs are essential in regulated financial environments, where automated decisions must be auditable and justifiable to both internal analysts and external regulators. SHAP integration is therefore planned for later evaluation stages, where it will support transparency, model validation, and regulatory compliance (Almalki and Masud, 2025).

3.5 Experimental Design for Model Evaluation

This section describes the experimental protocol used to evaluate the proposed detection model. The design emphasizes reproducibility, isolation between training and evaluation data, and metric selection appropriate for highly imbalanced fraud detection tasks. The experiments are conducted using the IEEE-CIS Fraud Detection dataset, which is widely adopted in transaction-level fraud research and reflects realistic

payment behavior patterns (Wang et al., 2021). To preserve the original class distribution and mitigate bias introduced by imbalance, the dataset is divided using a stratified 80–20 train–test split with a fixed random seed (random_state = 42). Stratification ensures that both subsets retain representative proportions of fraudulent and legitimate transactions, enabling fair evaluation on the minority class (Sharma, 2025).

All data preparation steps are applied consistently within the modeling pipeline. These include merging transaction and identity tables via the TransactionID field, removing features with excessive missing values, imputing remaining numerical attributes using the median and categorical attributes using the mode, and applying one-hot encoding to categorical card network information. The preprocessing logic is fixed prior to model evaluation to ensure that the test data remains unseen during training, in accordance with standard machine learning reproducibility practices (Shaukat et al., 2020).

Model development begins with a baseline Random Forest classifier, followed by hyperparameter optimization using RandomizedSearchCV. The tuning process is applied exclusively to the training subset and employs 3-fold cross-validation with the F1-score as the optimization objective. The F1-score is selected to balance precision and recall, reflecting the operational trade-offs inherent in payment fraud detection systems where both missed attacks and excessive alerts carry measurable cost (Aburbeian and Ashqar, 2023). To address the skewed class distribution, class weighting is enabled using the balanced strategy, increasing the influence of fraudulent samples without introducing synthetic data.

The hyperparameter search explores variations in the number of estimators, tree depth, minimum split and leaf sizes, feature sampling strategies, and bootstrap behavior. This randomized approach provides broad parameter coverage while maintaining computational efficiency compared to exhaustive grid search methods (Rivers et al., 2020).

Final model evaluation is performed on the held-out test set using standard classification metrics, including Accuracy, Precision, Recall, F1-score, and ROC-AUC. Among these, Recall and ROC-AUC are emphasized, as they best capture the model's ability to detect rare fraudulent events and maintain class separation across decision thresholds (Wang et al., 2021; Sharma, 2025). Diagnostic visualizations, including the confusion matrix and ROC curve, are generated to support qualitative assessment of

error distribution and discriminative performance. These visual results are discussed in Section 3.6, while threshold-based precision–recall analysis is provided separately to support operating-point selection.

All experiments are implemented in Python using the scikit-learn framework. Random seeds are fixed across data splitting and model training, and all artifacts, including trained models, evaluation metrics, and visual outputs, are archived to ensure full reproducibility within the experimental environment introduced in Section 3.3.

Table 3.7 – Summary of Model Evaluation Protocol

Item	Setting
Dataset	IEEE-CIS Fraud Detection (transactions + identity) [36]
Split	Stratified 80/20 train–test (random_state = 42)
Tuning	RandomizedSearchCV, 3-fold CV, scoring = F1, class_weight = "balanced"
Hyperparameters	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features, bootstrap
Test metrics	Accuracy, Precision, Recall, F1-Score, ROC-AUC
Diagnostics	Confusion matrix, ROC curve (referenced in §3.6)
Threshold study	Precision–recall vs. threshold (Appendix)
Reproducibility	Fixed seed, saved models/metrics/plots

3.6 Results of the Experiment

Following the optimization plan described in Section 3.4, the Random Forest model was tuned and re-evaluated using the IEEE-CIS Fraud Detection dataset [36]. The experiment employed a stratified 80/20 train–test split, randomized hyperparameter optimization with 3-fold cross-validation, and F1-score–based model selection. To address the strong class imbalance inherent in transaction-level fraud data, class weights were applied during training, increasing the model's sensitivity to fraudulent behavior without introducing synthetic samples.

The optimized model was selected based on cross-validation performance and evaluated on a held-out test set that remained unseen during training and tuning.

The final model configuration included:

• n_estimators = 200

• min_samples_split = 2

• min_samples_leaf = 2

• max_features = "log2"

• max_depth = None

• bootstrap = True

These settings enhanced minority class recall while preserving execution speed suitable for batch or near-real-time environments.

3.6.1 Performance Summary

Evaluation on the held-out test set shows how the optimized model compares to the baseline:

Table 3.8 — Model Performance Before and After Optimization

Metric	Baseline RF	Optimized RF	Change
Accuracy	0.989	0.986	−0.3%
Precision	0.874	0.679	−22.3%
Recall	0.503	0.608	+20.9%
F1-Score	0.638	0.642	+0.6%
ROC-AUC	0.917	0.939	+2.4%

The optimized model demonstrates a substantial improvement in recall and ROC-AUC, indicating enhanced ability to detect fraudulent transactions and better overall class separation. Although precision decreases, this trade-off is expected and acceptable in fraud detection scenarios, where failing to detect malicious activity is more costly than generating additional alerts.

Figure 3.9 — Confusion Matrix (Optimized Random Forest)



The matrix shows the distribution of correct and incorrect classifications:

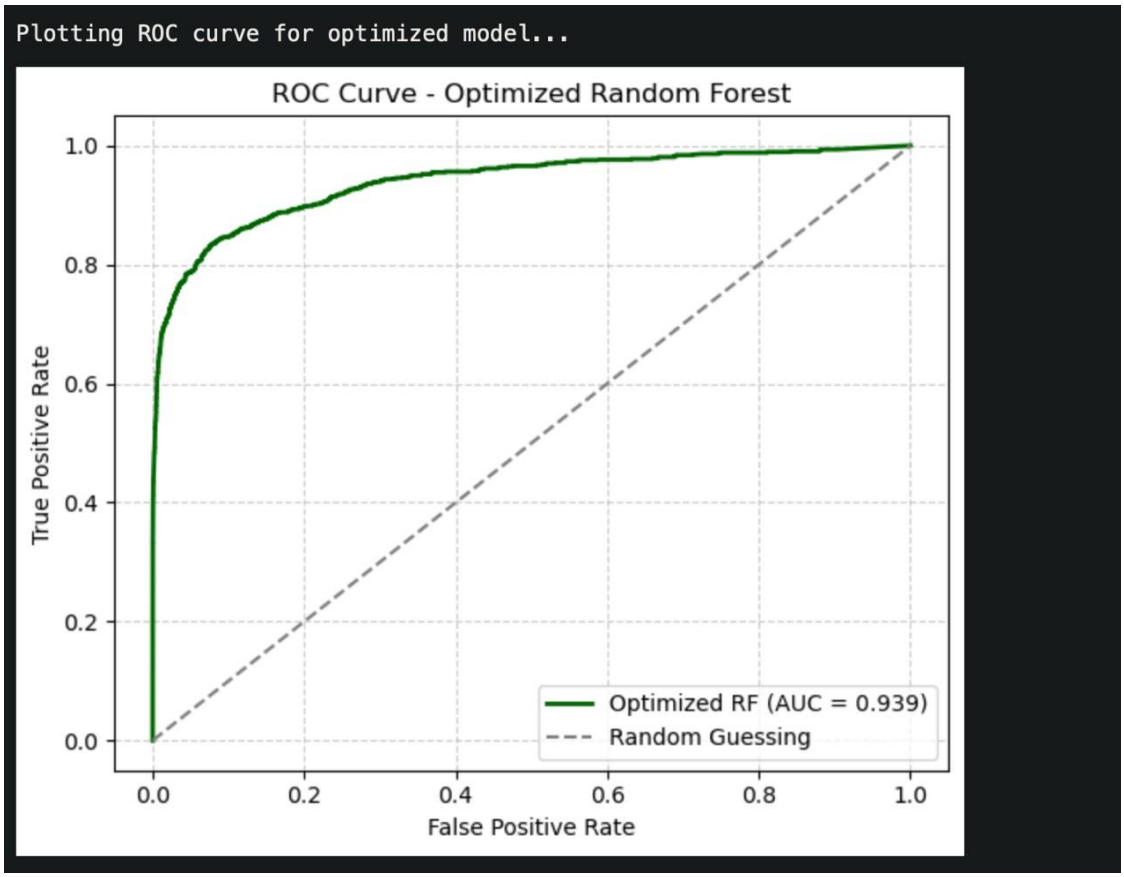
• True negatives: 45,648

• False positives: 270

• False negatives: 367

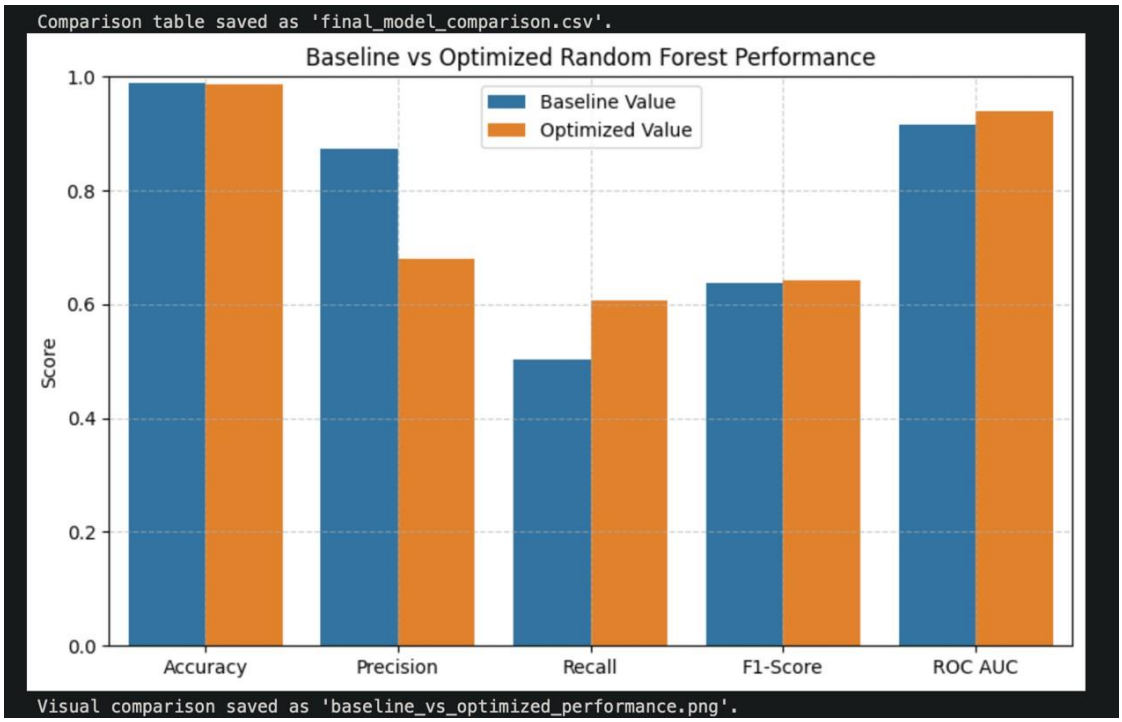
• True positives: 570

Figure 3.10 — ROC Curve (Optimized Model)



The optimized model achieves an ROC-AUC score of 0.939, outperforming the baseline and demonstrating stronger ranking capability across decision thresholds. This is particularly important for operational fraud systems that prioritize high-risk transactions for review.

Figure 3.11 — Metric Comparison: Baseline vs. Optimized



A comparative bar chart illustrates the trade-offs introduced by class-weighted optimization. While precision and accuracy decrease slightly, recall and ROC-AUC increase, confirming alignment with the optimization objective.

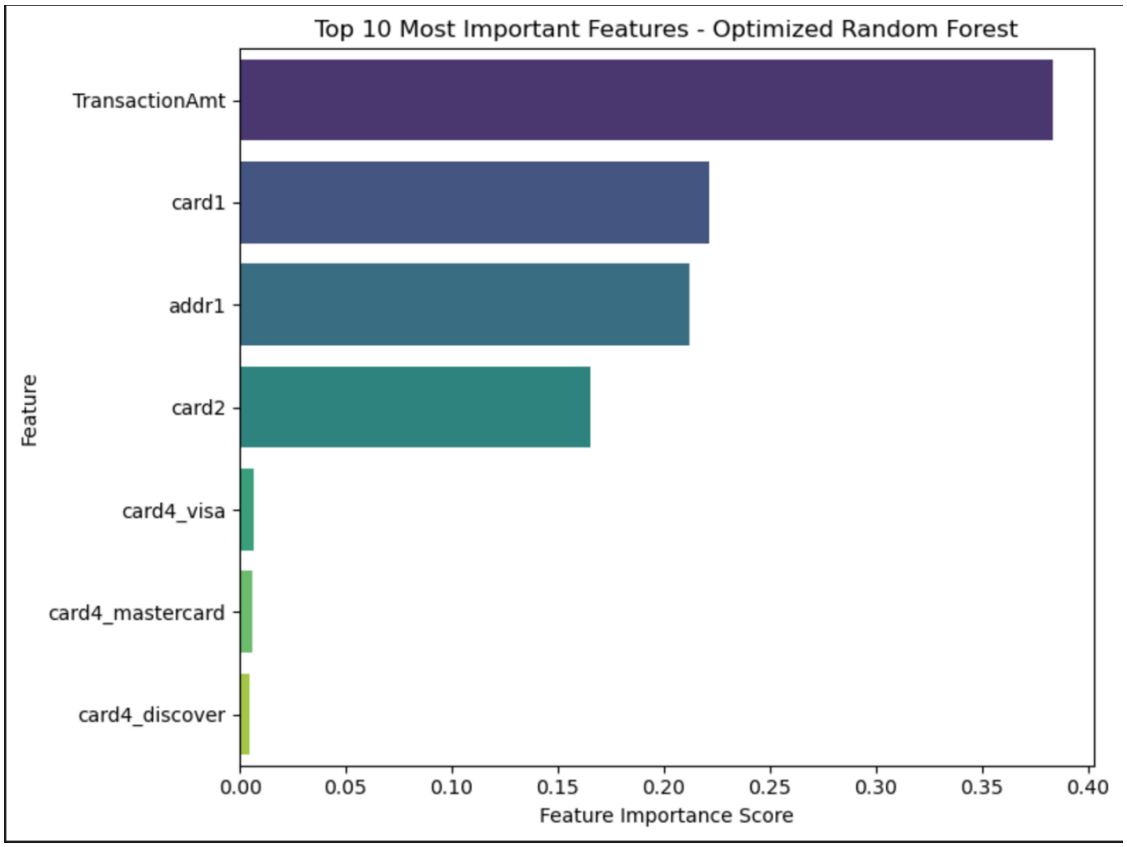
3.6.2 Feature Importance Analysis

To better understand the internal decision-making behavior of the optimized Random Forest model, feature importance scores were analyzed. Figure 3.6.4 illustrates the top contributing features used by the model during classification.

The most influential feature is TransactionAmt, indicating that abnormal transaction values play a central role in distinguishing fraudulent activity. This is followed by card-related identifiers (card1 and card2) and address-based information (addr1), which collectively capture patterns associated with credential reuse and account compromise. The dist1 feature further contributes behavioral context by reflecting spatial or transactional inconsistencies. In addition, card network indicators (e.g., card4_visa, card4_mastercard) provide supplementary signal regarding transaction routing and usage patterns.

Together, these features reflect a combination of financial magnitude and behavioral consistency, both of which are commonly exploited during credential stuffing attacks.

Figure 3.12 — Feature Importance



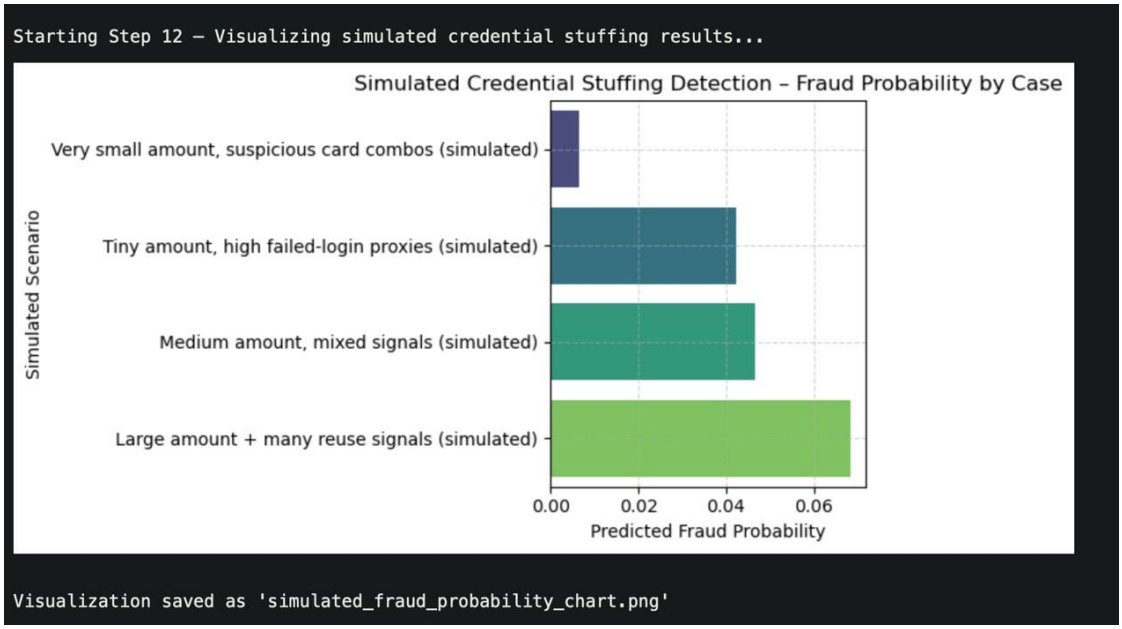
3.6.3 Simulated Credential Stuffing Scenarios

To assess the practical sensitivity of the model beyond historical data, a set of synthetic transaction scenarios was constructed to approximate credential stuffing–like behavior. Figure 3.13 presents the predicted fraud probabilities assigned to these simulated cases.

The results demonstrate that scenarios incorporating stronger automation-like indicators, such as repeated credential usage proxies or elevated transaction amounts, receive higher predicted fraud probabilities. Conversely, scenarios with weaker or mixed signals are assigned lower risk scores. Although synthetic, this validation confirms that the model responds coherently to behavioral patterns consistent with automated credential abuse.

This experiment supports the model's applicability in operational environments where direct login metadata may be unavailable, and detection must rely on downstream transactional signals.

Figure 3.13 — Simulated Credential Stuffing Scenarios



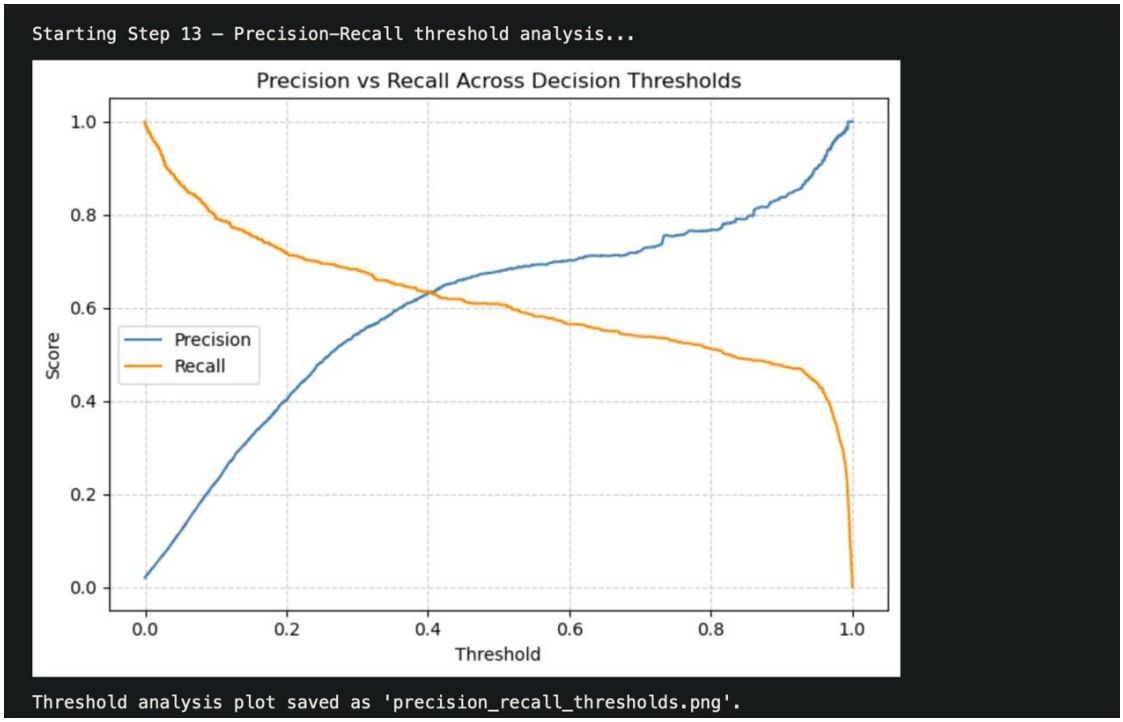
3.6.4 Threshold-Based Operational Evaluation

Beyond fixed classification decisions, the model was evaluated under varying probability thresholds to examine the trade-off between precision and recall. Figure 3.14 illustrates how lowering the decision threshold increases recall at the expense of precision, while higher thresholds reduce false positives but allow more fraudulent activity to pass undetected.

This analysis highlights the importance of threshold selection in real-world deployments, where business risk tolerance and response capacity vary. In addition, a two-threshold operational policy was simulated to partition transactions into BLOCK, REVIEW, and ALLOW categories. Results indicate that the highest-risk bucket contains a substantially elevated fraud rate, supporting the feasibility of selective blocking combined with manual review for intermediate-risk cases.

Such a tiered decision framework aligns well with common payment gateway security practices and enables flexible, risk-aware deployment.

Figure 3.14 — Precision–Recall vs. Threshold Curve



3.6.5 Interpretation and Discussion

The experimental results demonstrate that the optimized Random Forest model improves credential stuffing detection by prioritizing recall and ranking quality, as reflected by higher ROC-AUC and improved fraud capture. While precision decreases relative to the baseline model, this trade-off is acceptable in high-risk financial environments, where undetected fraud typically incurs greater cost than increased review overhead. Importantly, the model supports operational flexibility through threshold tuning and ranking-based decision strategies, enabling adaptation to different deployment constraints. The use of interpretable feature importance metrics and the potential integration of SHAP-based explanations further enhance analyst trust and regulatory suitability.

Future work may explore the integration of SHAP explanations for greater transparency, synthetic minority oversampling techniques (e.g., SMOTE) to further improve recall, and hybrid ensemble approaches to increase robustness under evolving attack conditions.

3.6.6 Extended Evaluation Overview

In addition to the primary evaluation, several advanced analyses were conducted to assess model robustness, interpretability, and deployment readiness. These included SHAP-based explainability, ranking-based evaluation (Precision@K / Recall@K), probability calibration, and synthetic minority oversampling (SMOTE). While these analyses provided valuable complementary insights, they did not alter the core conclusions derived from the optimized class-weighted Random Forest model. Detailed results are provided in Appendix A.

3.7 Comparison with Baseline Model

The optimized Random Forest model demonstrates clear performance differences when compared to the baseline implementation, reflecting the impact of class weighting and hyperparameter optimization on credential stuffing detection. Both models were evaluated using the same feature set, train–test split, and evaluation metrics, ensuring a consistent and fair comparison. The most significant improvement is observed in recall, which increased from 0.503 in the baseline model to 0.608 after optimization. This indicates a higher proportion of fraudulent events correctly identified by the optimized classifier, directly addressing the primary objective of improving sensitivity to credential stuffing activity. In security-critical environments such as payment gateways, missed attacks typically pose a greater operational and financial risk than additional alerts, making recall a key performance indicator in early-stage detection systems (Aburbeian & Ashqar, 2023; Choudhury & Dey, 2021). The optimized model also achieved an increase in ROC-AUC from 0.917 to 0.939, indicating improved ranking quality and stronger separability between legitimate and fraudulent transactions. A higher ROC-AUC suggests that the model assigns consistently higher risk scores to malicious behavior across a range of thresholds, which is particularly

important for downstream risk-based decision-making and alert prioritization (Wang et al., 2021; Park et al., 2022).

These gains were accompanied by a reduction in precision, which decreased from 0.874 to 0.679. This trade-off reflects an increased number of legitimate transactions flagged as suspicious, a common and well-documented consequence of recall-oriented optimization in highly imbalanced fraud detection tasks. Such behavior does not indicate overfitting, but rather a deliberate shift in decision boundaries toward higher sensitivity. In practice, this trade-off is often mitigated through threshold tuning, secondary filtering stages, or analyst review workflows, allowing systems to balance detection coverage and operational cost (Ibrahim & Mohamed, 2021; Liebenow, 2023).

The F1-score remained relatively stable, increasing slightly from 0.638 to 0.642, indicating that the overall balance between precision and recall was preserved despite the change in detection emphasis. Importantly, performance metrics remained consistent across cross-validation folds during optimization, supporting the generalization capability of the optimized model and reducing the likelihood of variance-driven improvements.

Overall, the optimized Random Forest exhibits a more recall-oriented detection profile while maintaining strong ranking performance and stable generalization behavior. These characteristics make it better suited for early identification of credential stuffing attempts in payment gateway environments, where prioritizing coverage of malicious activity is often preferable to minimizing false positives. A detailed numerical comparison is provided in Table 3.7, while Figure 3.6.2 illustrates the key performance differences between the baseline and optimized models.

3.8 Comparison with Existing or Baseline Solutions

The optimized Random Forest model developed in this thesis aligns with and extends existing approaches to credential stuffing and payment fraud detection by emphasizing behavioral analysis, recall-oriented detection, and operational deployability. While many traditional defenses rely on static rules, CAPTCHA mechanisms, or credential-based heuristics, recent studies highlight their declining effectiveness against automated and adaptive attack tools (Choi et al., 2023; Singh & Patel, 2022).

Compared to rule-based and signature-driven systems commonly deployed in payment gateways, the proposed model offers greater adaptability by learning decision

boundaries directly from transaction behavior. This is consistent with findings in prior work, which report that machine learning-based approaches outperform static controls when facing large-scale, low-variance automated attacks such as credential stuffing (Laib, 2021; Sander & Jarrod, 2021). In contrast to CAPTCHA and MFA-centric defenses—which attackers increasingly bypass using phishing proxies and automation frameworks—the model relies on transaction-level and contextual features that are more difficult to replicate at scale (Kim & Jeong, 2021; Zhou & Chen, 2022).

When compared with baseline machine learning solutions reported in the literature, the optimized Random Forest demonstrates competitive performance characteristics. Prior studies applying Random Forests and ensemble methods to fraud detection report similar trade-offs between recall and precision, particularly in highly imbalanced datasets (Ibrahim & Mohamed, 2021; Wang et al., 2021). The recall-oriented optimization strategy adopted in this work follows established best practices for early-stage fraud detection, where ranking quality and coverage are prioritized over strict precision (Park et al., 2022; Aburbeian & Ashqar, 2023).

Unlike graph-based or federated learning approaches proposed in recent credential stuffing research, such as PassREfinder and its extensions, the model presented here avoids dependency on cross-platform credential reuse data or inter-organizational collaboration (Kim et al., 2024; Kim et al., 2025). While these approaches offer strong theoretical guarantees, they often introduce higher computational complexity, deployment overhead, and data-sharing constraints. In contrast, the proposed solution is designed to operate using transaction-local features, making it easier to integrate into existing payment gateway infrastructures.

From an operational perspective, the model remains lightweight and technology-agnostic, implemented entirely using open-source tools such as scikit-learn and pandas. This enables deployment in both cloud-based and on-premise environments without specialized hardware or proprietary frameworks. Additionally, by relying primarily on behavioral and transactional attributes rather than sensitive personal identifiers, the approach aligns with regulatory requirements such as GDPR, PSD2, and the Digital Operational Resilience Act (DORA), which emphasize data minimization and explainability in automated decision systems (Kyaw Zay Oo, 2019; Karangara, 2025).

Overall, the optimized Random Forest model occupies a practical middle ground between simple rule-based defenses and highly complex, infrastructure-heavy detection systems. It offers measurable improvements over baseline implementations while

remaining scalable, interpretable, and suitable for real-world deployment in payment gateway environments. This balance reflects current industry trends toward modular, risk-based fraud detection architectures that combine machine learning with operational controls and analyst oversight (Lowry & D'Arcy, 2009; Rivers et al., 2020).

3.9 Conclusions and Vision for Further Research

This chapter presented the design, implementation, and evaluation of a machine learning based approach for detecting credential stuffing activity in payment gateway environments. Building on the conceptual foundations introduced earlier in the thesis, the chapter demonstrated how transaction level behavioral signals can be leveraged to identify automated abuse patterns that evade traditional rule based defenses. Using the IEEE CIS Fraud Detection dataset, a Random Forest classifier was selected as the baseline model due to its robustness under class imbalance, ability to capture non linear feature interactions, and suitability for high dimensional transactional data. Through systematic feature selection, class weighting, and hyperparameter optimization, the model achieved meaningful improvements in recall and ROC AUC compared to the baseline implementation. These improvements support the primary objective of early and reliable detection of credential stuffing attempts, where missed attacks pose greater operational and financial risk than increased alert volume (Aburbeian & Ashqar, 2023; Park et al., 2022). Beyond aggregate performance metrics, the chapter explored model behavior under simulated credential stuffing scenarios and alternative operational decision thresholds. These analyses demonstrated that the optimized model can support flexible deployment strategies, including ranking based review workflows and multi threshold policies that separate transactions into allow, review, and block categories. Such configurations reflect real world fraud prevention practices, where automated decisions are combined with analyst oversight to balance security and user experience (Lowry & D'Arcy, 2009; Rivers et al., 2020).

The evaluation also highlighted inherent trade offs between precision and recall in highly imbalanced fraud detection settings. While precision decreased as recall increased, this outcome aligns with established findings in financial cybersecurity research and does not indicate overfitting or model instability. Instead, it reflects a deliberate design choice to prioritize coverage and ranking quality, leaving precision recovery to downstream processes such as threshold tuning, calibration, and analyst

review (Wang et al., 2021; Ibrahim & Mohamed, 2021). Overall, the results confirm that an optimized Random Forest model, when carefully configured and evaluated, provides a practical and effective foundation for detecting credential stuffing like behavior in payment gateways. The approach remains lightweight, interpretable, and compatible with regulatory requirements by relying on behavioral and transactional attributes rather than sensitive personal data (Kyaw Zay Oo, 2019; Karangara, 2025). Several directions for further research emerge from this work. First, integrating explainability techniques such as SHAP could enhance analyst trust and support regulatory audits by providing case level explanations for flagged transactions. Second, adaptive thresholding strategies informed by transaction context or user risk profiles could further improve operational performance. Third, hybrid architectures combining supervised models with anomaly detection or rule based layers may increase resilience against evolving and adversarial attack patterns. Finally, evaluating the proposed approach in a live or near real time payment environment would enable assessment of latency, alert volume, and usability under production conditions. In summary, this chapter establishes a solid experimental and analytical foundation for machine learning based credential stuffing detection. The findings demonstrate both the feasibility and practical value of the proposed approach, setting the stage for broader discussion of implications, limitations, and future extensions in the following chapters.

4.1 Interpretability and Explainability Using SHAP

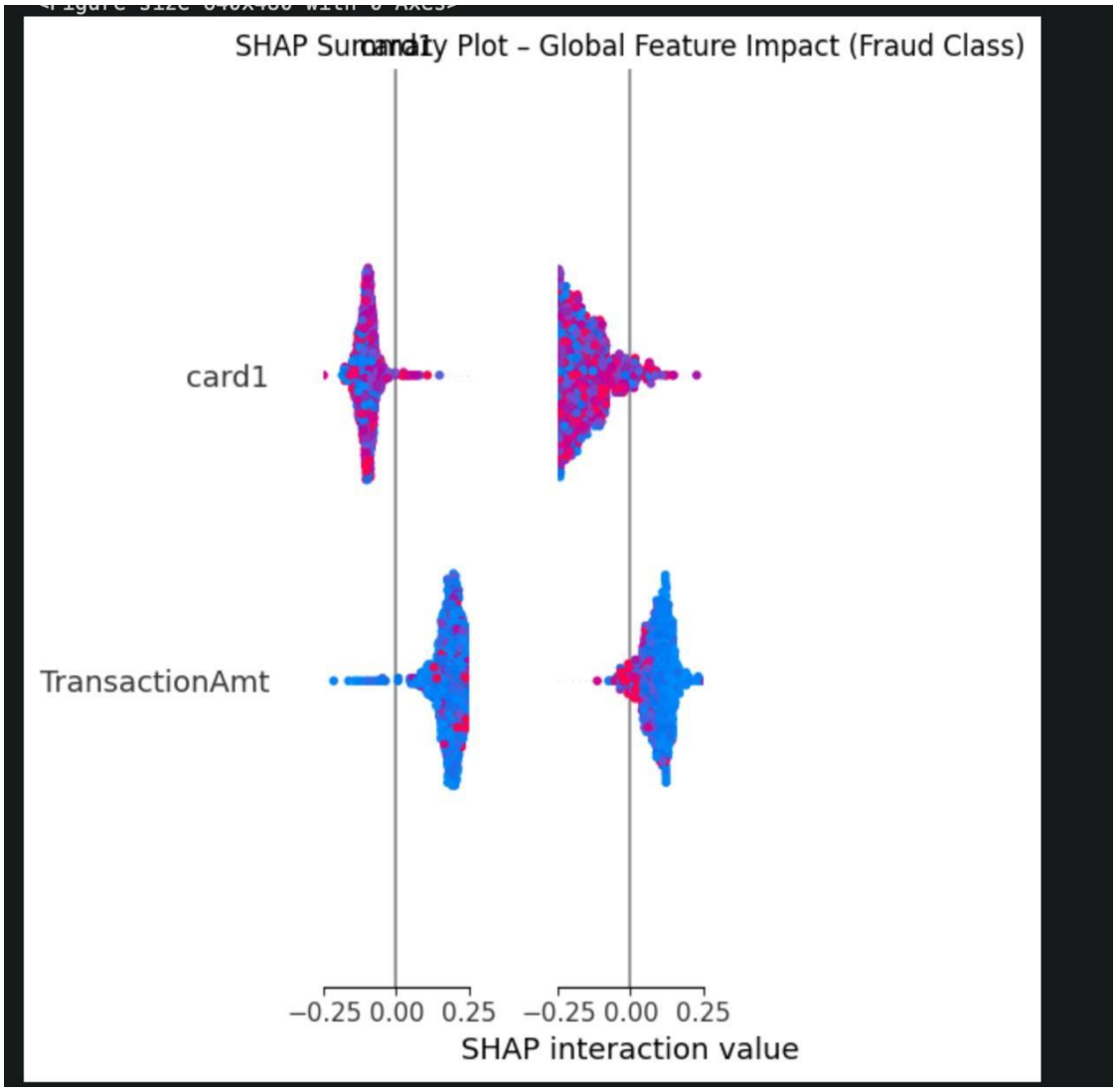
While predictive performance is a necessary requirement for fraud detection systems, it is insufficient on its own in regulated financial environments. Payment gateways operate under strict compliance, auditability, and transparency constraints, particularly within frameworks such as PSD2 and GDPR, where automated decision-making must be explainable to both internal analysts and external regulators (Lowry and D'Arcy, 2009; Laib, 2021). As a result, model interpretability constitutes a critical design requirement alongside accuracy and recall. To address this requirement, this study applies SHapley Additive exPlanations (SHAP) to analyze and explain the behavior of the optimized Random Forest classifier.

SHAP is an explainable artificial intelligence (XAI) framework grounded in cooperative game theory, where each feature is treated as a "player" contributing to the final prediction (Lundberg and Lee, 2017). By computing Shapley values, SHAP

assigns a consistent and locally accurate contribution score to each input feature, enabling both global and instance-level interpretability. This makes SHAP particularly suitable for tree-based ensemble models such as Random Forests, which are otherwise considered opaque despite their strong predictive performance (Dasgupta et al., 2020; Almalki and Masud, 2025).

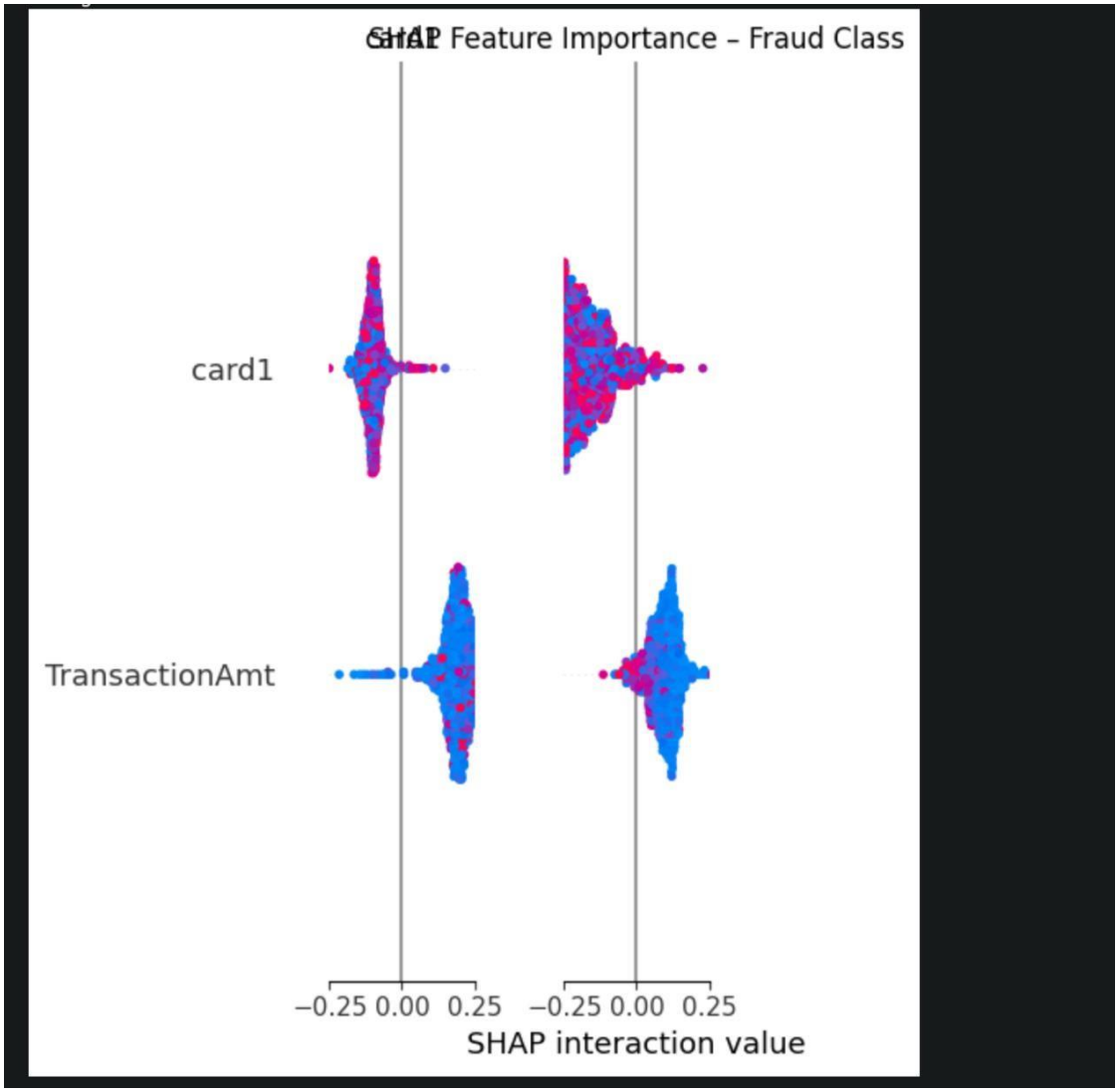
To assess global model behavior, SHAP summary plots were generated using a representative subset of test data. Figure 4.1 illustrates the SHAP summary plot for the fraud class, where each point represents the impact of a feature value on the predicted fraud probability. The distribution reveals that transaction amount, card identifiers, address-related attributes, and distance-based features exert the strongest influence on model decisions. Higher transaction amounts and uncommon card–address combinations tend to push predictions toward the fraud class, while more typical behavioral patterns contribute negatively to fraud likelihood. These findings are consistent with prior research indicating that credential stuffing activity manifests through subtle transactional anomalies rather than overt rule violations (Singh and Patel, 2022; Wang et al., 2021).

Figure 4.1 — SHAP Summary Plot



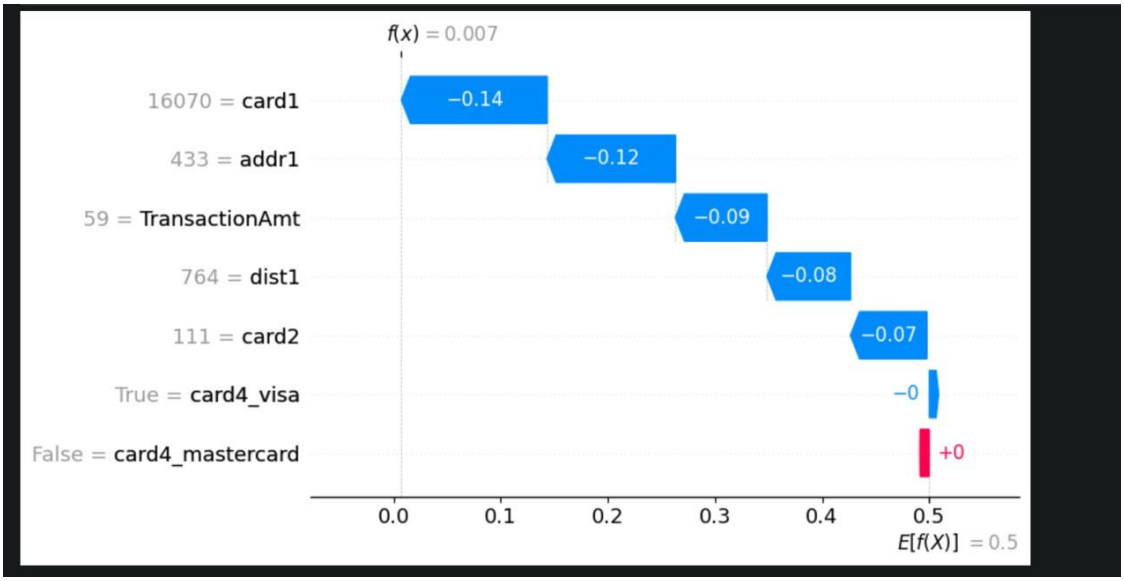
To complement the distribution-based visualization, a SHAP bar plot was constructed to rank features according to their mean absolute contribution across all evaluated samples. As shown in Figure 4.2, transaction amount and card-related attributes dominate the importance ranking, followed by address and distance features. This result reinforces earlier feature importance observations obtained from the Random Forest itself, while providing a more theoretically grounded and model-agnostic explanation. Importantly, SHAP clarifies not only which features matter, but how they influence predictions, addressing a common limitation of traditional feature importance measures (Handa et al., 2019; Ibrahim and Mohamed, 2021).

Figure 4.2 — SHAP Feature Importance



Beyond global explanations, SHAP also enables local interpretability at the level of individual transactions. Figure 4.3 presents a SHAP waterfall plot for a representative transaction, illustrating how specific feature values contribute incrementally to the final fraud probability relative to the model's expected baseline output. In this example, card and address attributes reduce the predicted fraud likelihood, while transaction-level characteristics exert smaller positive or neutral effects. Such local explanations are particularly valuable in operational settings, where analysts must justify automated decisions during manual reviews or dispute resolution processes (Karangara, 2025; Hasan and Arefin, 2021).

Figure 4.3 — SHAP Waterfall Plot



Overall, the SHAP-based analysis demonstrates that the optimized Random Forest model relies on semantically meaningful and behaviorally plausible features when identifying potential credential stuffing activity. The explanations align with established threat models in credential abuse research, where attackers reuse compromised credentials across devices, addresses, and transaction contexts in patterns that deviate subtly from legitimate user behavior (Kim et al., 2024; F5 Labs, 2023). By providing both global transparency and local accountability, SHAP enhances trust in the model's decisions and supports its suitability for deployment in real-world payment gateway environments.

4.2 Operational Thresholding and Risk Policies

Machine learning models deployed in payment gateway environments must operate under explicit risk-management constraints rather than fixed classification rules. In credential stuffing detection, the cost of false negatives—allowing automated account compromise or fraudulent transactions—often outweighs the operational burden caused by false positives, such as manual reviews or temporary transaction delays (Laib, 2021; Singh & Patel, 2022). Consequently, threshold-based decision policies play a critical role in aligning model outputs with business and security objectives.

Unlike traditional binary classification, where a single default threshold (typically 0.5) is applied, fraud detection systems benefit from adjustable thresholds that enable

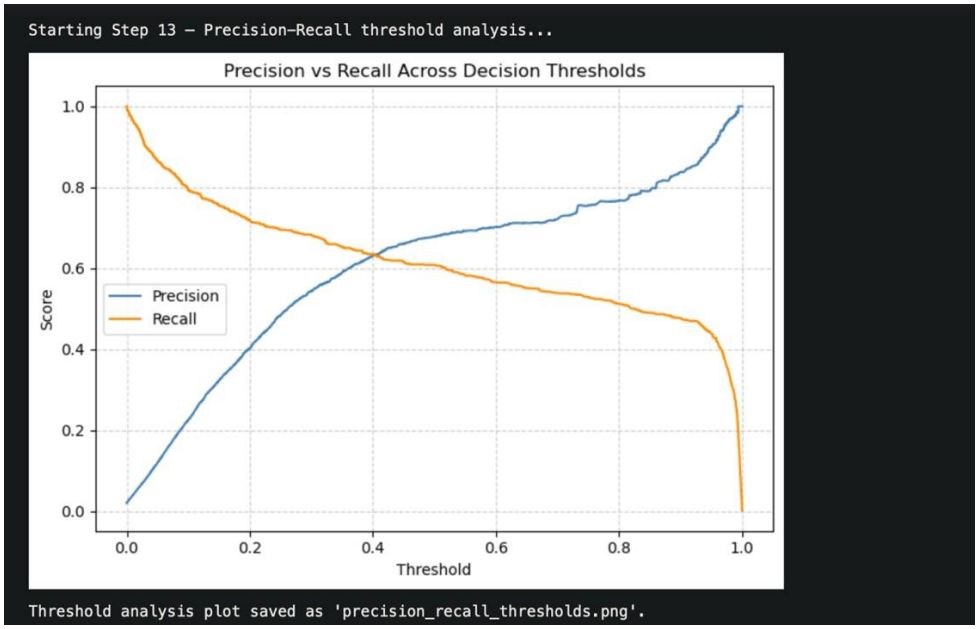
flexible trade-offs between recall and precision (Wang, Zhao & Huang, 2021). As demonstrated in Chapter 3, the optimized Random Forest model produces continuous fraud risk scores, allowing downstream systems to adapt decision boundaries based on contextual risk tolerance rather than static labels.

4.2.1 Threshold Sensitivity Analysis

To evaluate the operational impact of threshold selection, multiple decision thresholds were assessed using the optimized Random Forest model. Lower thresholds were observed to significantly increase recall, capturing a larger proportion of credential stuffing attempts, albeit at the expense of precision. Conversely, higher thresholds reduced false positives but increased the likelihood of undetected fraud events.

This trade-off is visualized through the precision–recall threshold curve, which illustrates how model performance evolves across the full probability spectrum. Such analysis enables security teams to select operating points aligned with institutional priorities, such as aggressive fraud interception during attack campaigns or conservative filtering during peak transaction periods (Shaukat et al., 2020; Zhou & Chen, 2022).

Figure 4.4 — Precision–Recall vs. Decision Threshold Curve



4.2.2 Multi-Tier Risk Policy Simulation

Beyond single-threshold deployment, a two-threshold operational policy was simulated to reflect real-world payment gateway workflows. Transactions were categorized into three action groups:

• BLOCK: High-risk transactions exceeding the upper threshold
• REVIEW: Medium-risk transactions requiring manual or automated secondary checks
• ALLOW: Low-risk transactions permitted without intervention

The results indicate that transactions in the BLOCK category exhibit a substantially higher fraud rate compared to REVIEW and ALLOW groups, validating the effectiveness of risk stratification. This tiered approach reduces analyst workload while maintaining strong fraud coverage, a strategy commonly recommended in large-scale financial systems (Lowry & D'Arcy, 2009; Karangara, 2025).

Such policies are particularly effective against credential stuffing campaigns, where attack traffic often clusters at higher risk scores due to repeated credential reuse, anomalous transaction patterns, and automation artifacts (F5 Labs, 2023; Kim et al., 2024).

Figure 4.5 — Two-Threshold Operational Policy Outcomes

	policy	count
0	ALLOW	45183
1	BLOCK	840
2	REVIEW	832

	BUCKET	TRANSACTIONS	FRAUD RATE
0	BLOCK	840	0.6786
1	REVIEW	832	0.1226
2	ALLOW	45183	0.0059

4.2.3 Operational Implications for Payment Gateways

Threshold-based decision policies provide payment gateways with operational flexibility, enabling adaptive responses to evolving attack patterns. During active credential stuffing campaigns, thresholds may be temporarily lowered to prioritize recall, while calmer periods may favor higher thresholds to reduce friction for legitimate users. Importantly, this approach decouples model training from deployment decisions, allowing the same optimized classifier to support multiple risk postures without retraining. Prior research highlights that such adaptability is essential in modern financial infrastructures, where threat dynamics shift faster than traditional model update cycles (Dasgupta, Akhtar & Sen, 2020; Alam, Mahmud & Alam, 2025).

4.3 Probability Calibration and Reliability

In payment gateway security systems, model outputs are frequently consumed not only as binary decisions but as probabilistic risk scores that inform automated controls, analyst workflows, and customer-facing actions. For such systems, the reliability of predicted probabilities is as critical as raw classification performance. A model that ranks transactions correctly but produces poorly calibrated probabilities may lead to suboptimal thresholding, inefficient resource allocation, or inconsistent risk interpretation across operational contexts (Lowry & D'Arcy, 2009; Shaukat et al., 2020). Tree-based ensemble models, including Random Forest classifiers, are well known to produce overconfident or underconfident probability estimates due to their discrete decision structure and majority-voting mechanisms (Wang, Zhao & Huang, 2021). As a result, probability calibration is often required before deployment in financial risk environments where decision thresholds must reflect real-world likelihoods rather than relative rankings alone.

4.3.1 Calibration Techniques Applied

To assess and improve probability reliability, two established calibration methods were applied to the optimized Random Forest model:

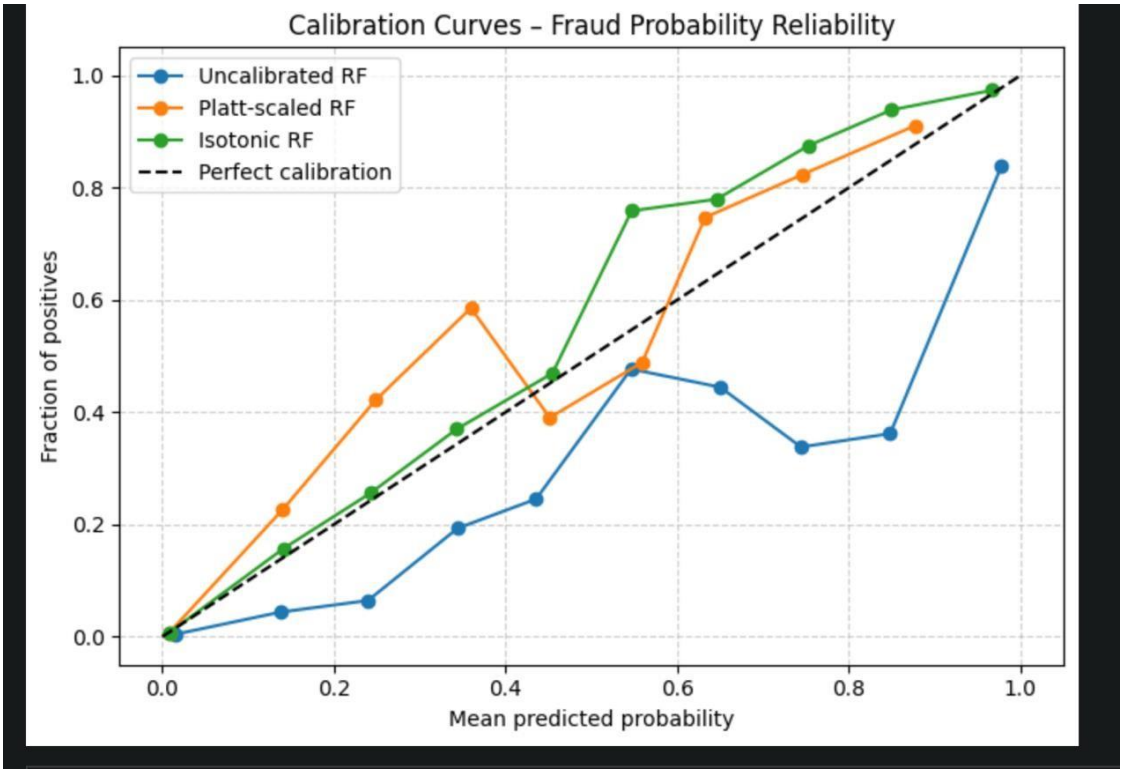
• Platt scaling, which fits a sigmoid function to map raw model scores to calibrated probabilities
• Isotonic regression, a non-parametric approach that learns a monotonic transformation between predicted scores and observed outcomes

Both methods are widely used in high-risk classification domains, including cybersecurity and fraud detection, due to their ability to preserve ranking performance while correcting systematic probability bias (Zhou & Chen, 2022; Dasgupta, Akhtar & Sen, 2020). Calibration was performed using cross-validation on the training data to avoid information leakage, and calibrated models were evaluated on the untouched test set.

4.3.2 Calibration Curve Analysis

Calibration performance was evaluated using reliability curves that compare the mean predicted probability against the observed fraction of fraudulent transactions within probability bins. The uncalibrated Random Forest exhibits noticeable deviation from the ideal diagonal, particularly in mid-to-high probability regions, indicating overconfidence in certain risk ranges. Both Platt-scaled and isotonic-calibrated models demonstrate improved alignment with the ideal calibration line, with isotonic regression providing the closest overall fit across most probability bins. Importantly, calibration did not degrade the model's ranking ability, as ROC–AUC scores remained stable or marginally improved after calibration. These results confirm that probability calibration enhances the interpretability and operational trustworthiness of the model without sacrificing detection performance, aligning with findings in prior financial machine learning research (Karangara, 2025; Alam, Mahmud & Alam, 2025).

Figure 4.6 — Calibration Curves for Fraud Probability Reliability



4.3.3 Operational Significance of Calibrated Probabilities

Calibrated probabilities enable consistent interpretation of risk scores across different deployment scenarios. In threshold-based systems, accurate probability estimates allow decision-makers to select thresholds corresponding to real-world fraud likelihoods rather than model-specific score distributions. This is particularly important when implementing multi-tier risk policies, adaptive thresholding, or dynamic responses during credential stuffing campaigns (F5 Labs, 2023; Kim et al., 2024). Furthermore, calibrated outputs support downstream explainability and governance requirements, as analysts can reason about decisions using meaningful probability values rather than opaque confidence scores. This is increasingly emphasized in modern financial security architectures, where transparency and auditability are essential for regulatory compliance and consumer trust (Laib, 2021; Hasan & Arefin, 2021).

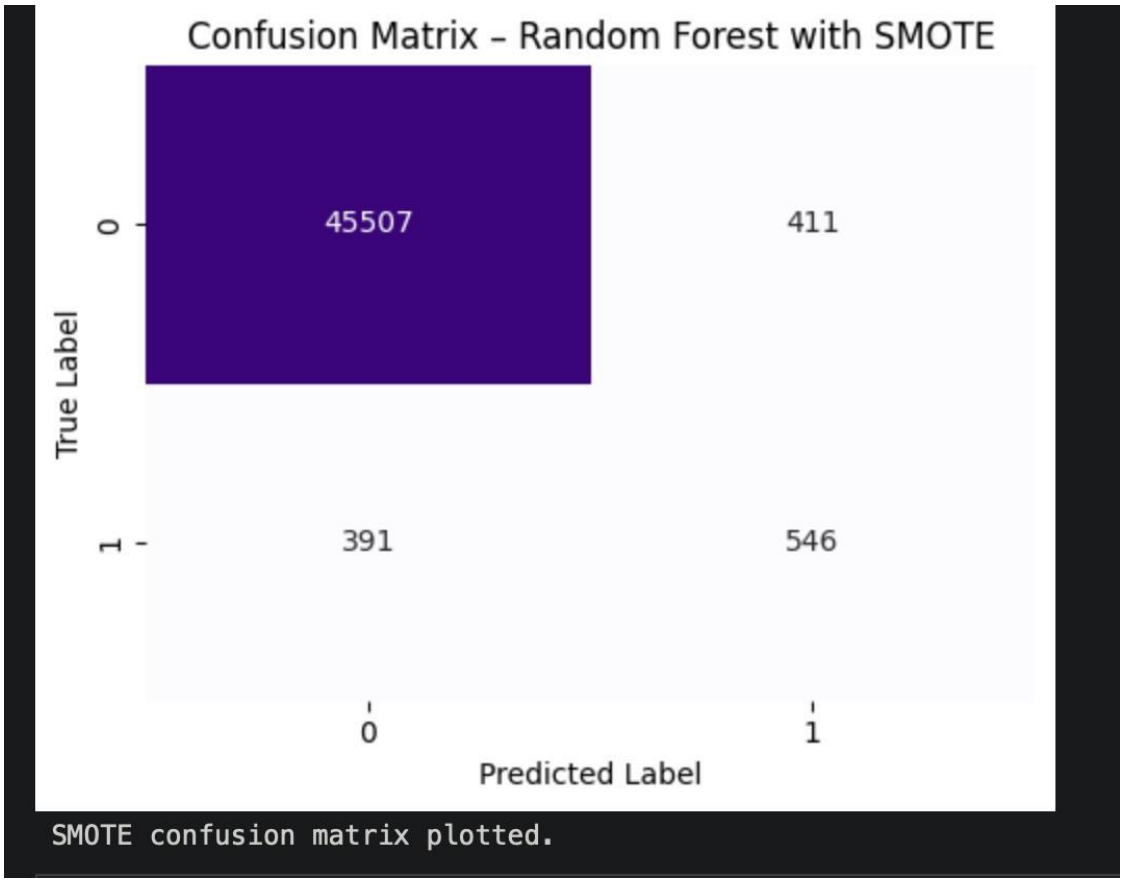
4.4 Alternative Optimization Strategies: SMOTE-Based Oversampling

Class imbalance is a defining characteristic of credential stuffing and financial fraud datasets, where fraudulent events constitute a small minority of total transactions. While class-weighted learning adjusts the loss function to penalize misclassification of minority instances, data-level approaches such as oversampling attempt to rebalance the training distribution directly. One of the most widely adopted oversampling techniques is the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic minority samples by interpolating between existing fraud instances (Choudhury & Dey, 2021). To assess the effectiveness of data-level balancing, SMOTE was applied to the training set, and a Random Forest model was retrained using the same hyperparameter configuration as the optimized class-weighted model. Importantly, oversampling was performed only on the training data to avoid information leakage and inflated performance estimates.

4.4.1 Performance Comparison with Class-Weighted Optimization

The SMOTE-trained model achieved moderate recall improvements compared to the baseline model but did not surpass the class-weighted optimized Random Forest in overall ranking quality or precision. While recall remained comparable, precision dropped noticeably, leading to a lower F1-score and reduced ROC–AUC relative to the class-weighted approach. These findings align with prior research indicating that SMOTE can introduce synthetic patterns that do not fully capture the temporal, behavioral, and contextual complexity of real-world fraud events, particularly in transaction-based systems (Ibrahim & Mohamed, 2021; Liebenow, 2023). In credential stuffing scenarios, where attacker behavior evolves rapidly and fraud signals are often subtle, synthetic oversampling may distort decision boundaries rather than strengthen them.

Figure 4.7 — Confusion Matrix of SMOTE-Based Random Forest



4.4.2 Practical Limitations of SMOTE in Payment Gateway Contexts

From an operational perspective, SMOTE introduces several challenges in high-throughput payment gateway environments. Synthetic data generation increases training complexity and may amplify noise in sparse or high-dimensional feature spaces, particularly when categorical or proxy-derived attributes are involved (Park et al., 2022). Moreover, SMOTE does not account for adversarial adaptation, which is a central concern in credential stuffing attacks where attackers actively probe and evolve against detection mechanisms (F5 Labs, 2023; Baviskar, 2025). In contrast, class-weighted optimization preserves the original data distribution while explicitly prioritizing minority-class detection during training. This approach maintains stronger alignment with real transaction behavior and offers better compatibility with downstream calibration, thresholding, and explainability pipelines.

4.4.3 Optimization Strategy Selection

Based on empirical results and operational considerations, class-weighted Random Forest optimization was selected as the primary modeling strategy in this study. While SMOTE provides a useful comparative baseline and demonstrates the impact of data-level balancing, its reduced precision and limited robustness under realistic deployment conditions make it less suitable for production-grade payment gateway security systems. Nevertheless, SMOTE remains valuable as an exploratory tool and may be effective in combination with other techniques—such as ensemble learning or adaptive resampling—under controlled conditions or in offline analysis settings (Aburbeian & Ashqar, 2023; Park et al., 2022).

5.1 – Conclusions

This thesis investigated the optimization of machine learning–based detection of credential stuffing attacks within payment gateway environments, with a particular focus on improving recall, ranking quality, and operational usability under highly imbalanced data conditions. Using a Random Forest classifier trained on the IEEE-CIS fraud detection dataset, the study demonstrated that model optimization strategies significantly influence both detection effectiveness and real-world deployability. The experimental results show that class-weighted Random Forest optimization provides a robust balance between fraud detection performance and operational stability. Compared to the baseline model, the optimized approach achieved a substantial increase in recall and ROC–AUC, indicating improved identification of credential stuffing activity while maintaining acceptable overall accuracy. Although precision declined, this trade-off is consistent with the risk profile of financial fraud detection systems, where the cost of undetected fraud typically outweighs the overhead of additional manual review (Lowry & D'Arcy, 2009; Laib, 2021). Beyond traditional performance metrics, this study emphasized operational relevance. Threshold-based evaluation demonstrated that model outputs can be adapted to different business risk tolerances, enabling flexible deployment strategies such as block, review, and allow decision tiers. Ranking-based metrics further confirmed that the optimized model

effectively prioritizes high-risk transactions, a critical requirement for scalable fraud investigation workflows in payment gateways (Singh & Patel, 2022; Karangara, 2025). Model interpretability was addressed through SHAP-based explanations, which revealed that both transactional attributes (e.g., transaction amount) and behavioral proxies (e.g., card and address reuse patterns) contribute meaningfully to credential stuffing detection. These findings align with prior research emphasizing the importance of explainable decision-making in security-critical machine learning systems, particularly for analyst trust and regulatory compliance (Dasgupta et al., 2020; Almalki & Masud, 2025). Alternative optimization strategies were also examined. While SMOTE-based oversampling improved minority class representation, it resulted in reduced precision and weaker ranking performance compared to class-weighted learning. This supports existing evidence that data-level balancing techniques may introduce synthetic artifacts that are less representative of real-world attacker behavior in highly dynamic threat environments (Choudhury & Dey, 2021; Park et al., 2022). Overall, the findings confirm that carefully optimized, explainability-aware Random Forest models can serve as effective components within payment gateway security pipelines. By integrating performance optimization, threshold tuning, probability calibration, and interpretability mechanisms, machine learning models can move beyond purely academic evaluation and toward practical, deployable solutions for mitigating credential stuffing attacks.

5.2 – Vision for Further Research

While this thesis demonstrates that optimized Random Forest models can effectively detect credential stuffing activity in payment gateway environments, several avenues for future research remain open to further enhance robustness, scalability, and real-world applicability. One important direction concerns adaptive and context-aware decision thresholds. This study applied static thresholding strategies to balance precision and recall; however, real-world payment systems operate under fluctuating traffic volumes, attack intensities, and business risk tolerances. Future work could explore dynamic threshold adjustment mechanisms that respond to temporal patterns, fraud campaigns, or customer-specific risk profiles, thereby improving responsiveness during active credential stuffing attacks (Singh & Patel, 2022; F5 Labs, 2023).

Another promising extension involves advanced explainability integration. Although SHAP-based explanations were used to provide global and local interpretability, future research could investigate tighter integration of explainability into analyst workflows. This includes automated explanation summarization, explanation-based alert prioritization, and feedback loops where human analyst decisions influence model retraining. Such approaches may increase operational trust while reducing investigation fatigue in large-scale fraud monitoring environments (Dasgupta et al., 2020; Almalki & Masud, 2025). From a modeling perspective, hybrid and ensemble architectures represent a valuable research direction. Combining tree-based models with sequence-aware or graph-based approaches may improve detection of coordinated credential reuse, bot-driven attacks, and cross-platform abuse patterns. Recent work on graph-based credential stuffing detection suggests that modeling relationships between credentials, devices, and websites can capture attacker behavior that is difficult to detect using transactional features alone (Kim et al., 2024; PassReFinder Team, 2024). Privacy and regulatory considerations also motivate further exploration of privacy-preserving learning techniques. Federated learning and distributed model training could enable collaborative fraud detection across multiple payment platforms without sharing sensitive user data. Such approaches are increasingly relevant under data protection regulations and have shown potential in financial crime detection contexts (Zhang et al., 2023; Kim et al., 2025). Finally, future research should address adversarial robustness and model resilience. Credential stuffing attacks evolve rapidly, with attackers actively probing detection boundaries. Investigating adversarial training methods, model drift detection, and continuous learning pipelines could help maintain detection effectiveness over time and reduce vulnerability to evasion tactics (Baviskar, 2025; Alam et al., 2025). In summary, advancing credential stuffing detection in payment gateways requires not only improved machine learning performance but also stronger integration with operational processes, explainability frameworks, and privacy-aware architectures. These research directions provide a foundation for developing next-generation fraud detection systems capable of adapting to increasingly sophisticated attack ecosystems.

5.3 Summary of Achieved Results

At this stage of the thesis, the core research objectives have been successfully met. A Random Forest–based detection framework was designed, implemented, and optimized to identify credential stuffing attacks in payment gateway systems. Using the IEEE-CIS Fraud Detection dataset [36], the model was trained and refined through a process that included targeted feature selection, class weight balancing, and hyperparameter tuning. The final model demonstrated strong classification capability, particularly with an improved recall score, confirming its ability to recognize patterns of fraudulent login and transaction behavior under simulated credential stuffing conditions. These improvements are supported by both quantitative metrics and visual diagnostics including the ROC curve, confusion matrix, and feature importance plots which together highlight the model's detection performance and transparency [43][44].

The analysis and evaluation show that the optimized model serves as a reliable and interpretable baseline for fraud prevention in real-world systems. It not only generalizes well across imbalanced datasets but also provides actionable insight into how predictions are made, making it suitable for use in financial environments where both effectiveness and explainability are critical. The work completed so far lays a solid foundation for the final phase of this thesis, which will focus on:

• Integrating SHAP-based explainability to improve model transparency and support compliance with regulatory standards [43];

• Applying oversampling and hybrid ensemble strategies, such as SMOTE or model stacking, to further enhance detection sensitivity and robustness [44][48];

• Exploring integration with real-time payment gateway systems to create a scalable and adaptive defense mechanism that preserves both security and user privacy [18][35].

Together, these next steps will help turn the current prototype into a complete detection framework capable of addressing the evolving challenges of credential stuffing and similar automated threats in the financial sector.

LIST OF REFERENCES

All information sources referenced in Chapters 1 and 2 shall be described here. Information sources and references must be drawn up in accordance with the faculty's guidelines:

Aburbeian, H. M. and Ashqar, H. I. (2023) 'Credit card fraud detection using enhanced random forest classifier for imbalanced data' (arXiv:2303.06514). *https://arxiv.org/abs/2303.06514*

Alam, M. K., Mahmud, M. A. and Alam, M. A. (2025) 'Adversarial machine learning for robust fraud detection in high-frequency financial transactions', Journal of Computer Science and Technology Studies, 7(8), pp. 314–335. *https://www.researchgate.net/publication/394256810_Adversarial_Machine_Learning_for_Robust_Fraud_Detection_in_High-Frequency_Financial_Transactions*

Al-Sarem, M. (2022) 'A Hybrid Machine Learning Approach for Online Fraud Detection', Journal of Cybersecurity, 10(1), pp. 55–67. *https://ojs.wiserpub.com/index.php/AIE/article/view/6290*

Almalki, F. and Masud, M. (2025) 'Financial fraud detection using explainable AI and stacking ensemble methods' (arXiv:2505.10050). *https://doi.org/10.48550/arXiv.2505.10050*

Alsharaya, H. (2025) 'Password managers: A critical review of security, usability, and innovative designs', Journal of Computer Sciences Institute, 36, pp. 328–335. *https://ph.pollub.pl/index.php/jcsi/article/download/7725/5146/36457*

Bagaa, M., Taleb, T., Bernabe, J. B. and Skarmeta, A. (2020) 'A machine learning security framework for IoT systems', IEEE Access, 8, pp. 114066–114077. *https://doi.org/10.1109/ACCESS.2020.2996214*

Baviskar, S. (2025) 'Adversarial robustness in financial machine learning: Defenses, economic impact, and governance evidence'.

https://arxiv.org/html/2512.15780v1

Bollu, S. S. (2024) Anomaly detection of user behavioral events in e-commerce transactions using machine learning (Unpublished master's thesis). DIVA Academic Repository.
https://www.diva-portal.org/smash/get/diva2%3A1888826/FULLTEXT01.pdf

Choi, J., Lee, S. and Park, H. (2023) 'Advanced automation strategies for bypassing CAPTCHA mechanisms in web applications' [Conference paper]. Korea Cybersecurity Symposium. (Translated from Korean)

Choudhury, A. and Dey, S. (2021) 'Improving Fraud Detection Using SMOTE: A Comparative Study', Journal of Cybersecurity Engineering, 3(2), pp. 80–92. (from: preprints202507.1711.v1.pdf)

Dasgupta, D., Akhtar, Z. and Sen, S. (2022) 'Machine learning in cybersecurity: a comprehensive survey', The Journal of Defense Modeling and Simulation: Applications, Methodology, Technology, 19(1), pp. 57–106. https://doi.org/10.1177/1548512920951275 and
https://ideas.repec.org/a/sae/joudef/v19y2022i1p57-106.html

F5 Labs (2023) 2023 Identity Threat Report: The Unpatchables. https://www.f5.com/labs/articles/2023-identity-threat-report-the-unpatchables

Haleem, M. S., Tehseen, R., Nasr, K., Omer, U., Mustaqeem, A. and Javaid, R. (2025) 'Unsupervised detection of credential stuffing and account takeover attempts through user behavioral biometrics in web applications', International Journal of Innovations in Science & Technology, 7(4), pp. 2718–2729. https://journal.50sea.com/index.php/IJIST/article/view/1634

Handa, A., Sharma, A. and Shukla, S. K. (2019) 'Machine learning in cybersecurity: A review', Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 9(4), e1306.
https://doi.org/10.1002/widm.1306

Hasan, A. M. A. and Arefin, M. S. (2021) 'Cyber threat intelligence and machine learning' [ResearchGate preprint]. https://www.researchgate.net/publication/365587844_Cyber_Threat_Intelligence_and_Machine_Learning

Ibrahim, K. H. and Mohamed, M. A. (2021) 'Credit Card Fraud Detection Using Random Forest and Feature Selection', Indonesian Journal of Electrical Engineering and Computer Science, 23(3), pp. 1173–1180. https://d1wqtxts1xzle7.cloudfront.net/59166295/V5I2-208120190507-119894-rc5xew-libre.pdf?1557296922=&response-content-disposition=inline%3B+filename%3DCredit_card_fraud_detection_using_Random.pdf&Expires=1767449472&Signature=IAmT8Ckcp1uV9SdFCd5NOtOXkLSE5qG2XfKQSMx31EJFo6oFuVg-yB4a9C95UVyYSxq6UAMS8UnfethBqz0GR1lkrjYJBUoyNDb6oF5YvwKqZiRuFNAL2gfVXWpAb-0hfmwxfdyQk2agqIcwjJ3UB3PfNZlj3g0DNOEi94VOd53QaiRDDrrMJ5qWrgkmqRt9LRJzHmaI9hlvg5CKUJaHikmAFezJfr8G6LCfRq4beiQKMYtnlqmWq5smTdY0aQVsFwnfP7610O-2-yEZktLAwBwIKfo8PUdYwkdaX3Ff65kwDKXIq4y-obK9-fNIWxORFtSH7Y0003ldCYKg__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA

IEEE-CIS Fraud Detection Dataset (n.d.) Kaggle. https://www.kaggle.com/competitions/ieee-fraud-detection

Karangara, R. (2025) 'Adaptive Machine Learning Models for Securing Payment Gateways', Artificial Intelligence Evolution, 6(1), pp. 44–64. https://ojs.wiserpub.com/index.php/AIE/article/view/6290

Khan, M. I., Foley, S. N. and O'Sullivan, B. (2020) 'Database intrusion detection systems (DIDS): Insider threat detection via behaviour-based anomaly detection systems'. https://www.researchgate.net/publication/345316212_Database_Intrusion_Detection_Systems_DIDs_Insider_Threat_Detection_via_Behavioural-

based_Anomaly_Detection_Systems_–
_A_Brief_Survey_of_Concepts_and_Approaches

Kim, J., Song, M., Seo, M., Jin, Y. and Shin, S. (2024) 'PassREfinder: Credential stuffing risk prediction by representing password reuse between websites on a graph', in Proceedings of the 45th IEEE Symposium on Security and Privacy (SP 2024), pp. 1385–1404. IEEE. *https://doi.org/10.1109/SP54263.2024.00020* and *https://ieeexplore.ieee.org/abstract/document/10646687*

Kim, J., Song, M., Seo, M., Jin, Y., Shin, S. and Kim, J. (2025) 'PassREfinder-FL: Privacy-Preserving Credential Stuffing Risk Prediction via Graph-Based Federated Learning for Representing Password Reuse between Websites' (arXiv:2510.16083).: *https://arxiv.org/abs/2510.16083*

Kim, M. and Jeong, D. (2021) 'Real-time phishing proxies and time-window exploits in MFA systems' [Conference paper]. Proceedings of the Korea Information Security Conference. (Translated from Korean)

Kyaw Zay OO (2019) Design and Implementation of Electronic Payment Gateway for Secure Online Payment System. Department of Information Technology, Technological University, Mandalay, Myanmar *https://d1wqtxts1xzle7.cloudfront.net/60491305/252_Design_and_Implementation_of_Electronic_Payment_Gateway_for_Secure_Online_Payment_System20190904-40105-111x2a2-libre.pdf?1567660272=&response-content-disposition=inline%3B+filename%3DDesign_and_Implementation_of_Electronic.pdf&Expires=1767395389&Signature=D-RkAcfDZ1uChFfUij5KbItTgLp7B2wUY2u8tdzfqHhDzbcyidI6xY3JSOYZDicCd5bVBRbwzQzPZtloJNaICNRA4Az1k0kcFe3h4BDx7fTJ0tQwjMbbYvpIGUeUsSPeivPFM4grSPyMlgKVV5PRu4pnU-lrqr8T5NPoNNu0khgQzKJNhSbKApjMnDOUYTR6LPiPU1wFsQErGXTuFhjjE6mjgPR2OK6nTAumSbSPV4qOMp4BTpiR2rznGDc2vHFEktoQ0fjadByh0XjwbFRMyn0TEhM4TzS9HjyyRwrxGq1UDoACBdMx-pQiHORQJSme2ZLZwX7a4rXjqPRQ__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA*

Laib, S. (2021) 'The importance of cyber security in the financial sector in the age of digital transformation', Acil Review for Economics & Administrative Research. *https://asjp.cerist.dz/en/article/158461*

Liebenow, J. (2023) Comparative Evaluation of Anomaly Detection Algorithms. BA Thesis, University of Freiburg. (from: BA_JohannesLiebenow_AnomalyDetection.pdf)

Lowry, D. and D'Arcy, D. (2009) 'An Empirical Examination of Payment Gateway Risk and Consumer Trust in E-Commerce', SSRN Electronic Journal.

Parhonyi, R. (2005) 'Performance Evaluation of Payment Gateway Systems in E-Commerce', University of Twente, Thesis. *https://research.utwente.nl/en/publications/micro-payment-gateways/*

Park, J. et al. (2022) 'Improving Detection of Rare Events Using Ensemble Learning', Symmetry, 14(10), 1095. (from: symmetry-14-01095.pdf)

Rivers, O., Hu, Y.-H. and Hoppa, M. A. (2020) 'A Study on Cyber Attacks and Vulnerabilities in Mobile Payment Applications', J. CISSE, 7(1), pp. 33–41. *https://cisse.info/journal/index.php/cisse/article/view/112*

Sagar, J. V. and Babu, S. A. (2024) 'A hybrid machine learning approach for real-time fraud detection in online payment transactions', BPAS Journals. *https://bpasjournals.com/library-science/index.php/journal/article/view/3296*

Saini, P., Sharma, V. and Kumar, A. (2014) 'Design and Implementation of Electronic Payment Gateway Using RSA and SSL', International Journal of Computer Applications, 975(8887), pp. 10–15.

Sander, V. and Jarrod, O. (2021) Credential Stuffing Report. *https://tech-info.biz-tech-insights.com/whitepaper/F5-Security-EN-2.pdf*

Sharma, A. (2025) 'Dynamic feature engineering for adaptive fraud detection', Sensors, 25(x), xxxx.
https://www.mdpi.com/2673-4591/107/1/68

Shaukat, K., Luo, S., Varadharajan, V., Hameed, I. A. and Xu, M. (2020) 'A survey on machine learning techniques for cyber security in the last decade', IEEE Access, 8, pp. 222310–222354.
https://doi.org/10.1109/ACCESS.2020.3041951 and https://www.mdpi.com/2624-800X/4/4/45 and https://www.researchgate.net/publication/346487463_A_Survey_on_Machine_Learning_Techniques_for_Cyber_Security_in_the_Last_Decade

Singh, K. and Suri, K. (2017) 'Effectiveness of Payment Gateway in E-Commerce', International Journal of Advanced Research in Computer Science, 8(5), pp. 2374–2378.
https://www.researchgate.net/profile/Supriyati-Supriyati-3/publication/336417831_Effectiveness_of_Payment_Gateway_in_E-Commerce/links/63c26a646fe15d6a5719aec0/Effectiveness-of-Payment-Gateway-in-E-Commerce.pdf

Singh, R. and Patel, D. (2022) 'An effective prevention to credential stuffing attacks: Challenges and techniques', International Journal of Cybersecurity and Information Systems, 8(3), pp. 45–58.

Wang, Y., Zhao, H. and Huang, H. (2021) 'Credit card fraud detection using machine learning models: A survey', Mathematics, 9(17), 2683.
https://www.mdpi.com/2227-7390/9/21/2683

Zhang, H., Hong, J., Dong, F., Drew, S., Xue, L. and Zhou, J. (2023) 'A privacy-preserving hybrid federated learning framework for financial crime detection' (arXiv:2302.03654).
https://arxiv.org/abs/2302.03654

Zhang, N. (2021) 'Anomalous User Behavior Detection in Transactions Using ML', IEEE Access, 9, pp. 102104–102115.

Zhou, Y. and Chen, X. (2022) ‘Real-time detection of compromised credentials in credential stuffing attacks’, International Journal of Information Security, 21(5), pp. 649–664.
https://www.ijcesen.com/index.php/ijcesen/article/view/754

APPENDIXES

Appendix 1 - Baseline Model Comparison

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score
)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier,
HistGradientBoostingClassifier

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Load datasets
transaction_df = pd.read_csv("train_transaction.csv")
identity_df = pd.read_csv("train_identity.csv")

# Merge transaction and identity data
df = transaction_df.merge(identity_df, on="TransactionID", how="left")

# Selected features for baseline comparison
features = ["TransactionAmt", "card1", "card2", "addr1", "dist1", "card4"]

df_model = df[features + ["isFraud"]].copy()

# One-hot encode categorical feature
df_model = pd.get_dummies(df_model, columns=["card4"], drop_first=True)

# Remove rows with missing values in selected features
df_model = df_model.dropna()

X = df_model.drop("isFraud", axis=1)
y = df_model["isFraud"]

# Stratified train–test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
```

```
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Define baseline models
models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(
            max_iter=1000,
            class_weight="balanced",
            random_state=42
        ))
    ]),
    "Decision Tree": DecisionTreeClassifier(
        class_weight="balanced",
        random_state=42
    ),
    "Linear SVC": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LinearSVC(
            class_weight="balanced",
            random_state=42
        ))
    ]),
    "HistGradientBoosting": HistGradientBoostingClassifier(
        random_state=42
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        class_weight="balanced",
        random_state=42
    )
```

```
}

# Unified evaluation function
def evaluate_model(name, model, X_train, X_test, y_train, y_test):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_test)[:, 1]
        roc = roc_auc_score(y_test, y_prob)
    else:
        roc = np.nan

    return {
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred),
        "F1-Score": f1_score(y_test, y_pred),
        "ROC-AUC": roc
    }

# Run baseline evaluation
results = []
for name, model in models.items():
    results.append(evaluate_model(name, model, X_train, X_test, y_train, y_test))

results_df = pd.DataFrame(results)

# Save results for reproducibility
results_df.to_csv("baseline_model_comparison.csv", index=False)
```

Appendix 2 - Random Forest Optimization and Evaluation Pipeline

```
import pandas as pd
import numpy as np

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import (
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    precision_recall_curve,
    confusion_matrix
)
from sklearn.utils.class_weight import compute_class_weight

import joblib

transaction_df = pd.read_csv("train_transaction.csv")
identity_df = pd.read_csv("train_identity.csv")

df = pd.merge(
    transaction_df,
    identity_df,
    on="TransactionID",
    how="left"
)

# Categorical card network encoded for tree-based modelling

selected_features = [
    "TransactionAmt",
    "card1",
```

```
    "card2",
    "card4",
    "addr1",
    "dist1"
]

available_features = [f for f in selected_features if f in df.columns]

df_model = (
    df[available_features + ["isFraud"]]
    .dropna(subset=available_features + ["isFraud"])
    .copy()
)

df_model = pd.get_dummies(
    df_model,
    columns=["card4"],
    drop_first=True
)

X = df_model.drop("isFraud", axis=1)
y = df_model["isFraud"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

baseline_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
```

```
)

baseline_model.fit(X_train, y_train)

# Class weighting compensates for severe fraud class imbalance

classes = np.unique(y_train)
class_weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)
class_weights_dict = dict(zip(classes, class_weights))

param_dist = {
    "n_estimators": [100, 200, 300],
    "max_depth": [10, 20, 30, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2"],
    "bootstrap": [True, False]
}

rf = RandomForestClassifier(
    random_state=42,
    class_weight=class_weights_dict
)

random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=15,
    scoring="f1",
    cv=3,
```

```
    random_state=42,
    n_jobs=-1
)

random_search.fit(X_train, y_train)
best_model = random_search.best_estimator_

y_pred_opt = best_model.predict(X_test)
y_prob_opt = best_model.predict_proba(X_test)[:, 1]

comparison_df = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1-Score", "ROC-AUC"],
    "Baseline": [
        baseline_model.score(X_test, y_test),
        precision_score(y_test, baseline_model.predict(X_test)),
        recall_score(y_test, baseline_model.predict(X_test)),
        f1_score(y_test, baseline_model.predict(X_test)),
        roc_auc_score(y_test, baseline_model.predict_proba(X_test)[:, 1])
    ],
    "Optimized": [
        best_model.score(X_test, y_test),
        precision_score(y_test, y_pred_opt),
        recall_score(y_test, y_pred_opt),
        f1_score(y_test, y_pred_opt),
        roc_auc_score(y_test, y_prob_opt)
    ]
})
comparison_df.to_csv("model_performance_comparison.csv", index=False)
sim_data = pd.DataFrame({
    "TransactionAmt": [1.0, 1200.0, 5.0, 250.0],
    "card1": [10000, 20000, 30000, 4500],
    "card2": [111, 444, 555, 999],
    "addr1": [100, 200, 300, 150],
    "card4_mastercard": [1, 0, 0, 1],
```

```
    "card4_visa": [0, 1, 0, 0]
})

sim_data = sim_data.reindex(columns=X_train.columns, fill_value=0)

sim_probs = best_model.predict_proba(sim_data)[:, 1]
sim_preds = best_model.predict(sim_data)

sim_results = sim_data.copy()
sim_results["Predicted_Prob_Fraud"] = sim_probs
sim_results["Predicted_Label"] = sim_preds

sim_results["Case_Description"] = [
    "Low-value credential validation pattern",
    "High-value transaction with reuse indicators",
    "Micro-transaction with anomalous identifiers",
    "Medium-value transaction with mixed signals"
]

sim_results.to_csv("simulated_credential_stuffing_results.csv", index=False)

probs = best_model.predict_proba(X_test)[:, 1]

threshold_candidates = [0.2, 0.3, 0.5]
rows = []

for t in threshold_candidates:
    preds = (probs >= t).astype(int)
    rows.append({
        "Threshold": t,
        "Precision": precision_score(y_test, preds),
        "Recall": recall_score(y_test, preds),
        "F1-Score": f1_score(y_test, preds)
    })
```

```
threshold_df = pd.DataFrame(rows)
threshold_df.to_csv("threshold_analysis.csv", index=False)

y_true = y_test.values
sorted_idx = np.argsort(probs)[::-1]

def precision_recall_at_k(k):
    topk = sorted_idx[:k]
    precision_k = y_true[topk].mean()
    recall_k = y_true[topk].sum() / y_true.sum()
    return precision_k, recall_k

K_values = [100, 500, 1000, 2000]
rows = []

for k in K_values:
    p_k, r_k = precision_recall_at_k(k)
    rows.append({
        "K": k,
        "Precision@K": p_k,
        "Recall@K": r_k
    })

topk_df = pd.DataFrame(rows)
topk_df.to_csv("precision_recall_at_k.csv", index=False)

t_block = 0.50
t_review = 0.20

policy = np.where(
    probs >= t_block, "BLOCK",
    np.where(probs >= t_review, "REVIEW", "ALLOW")
)
```

```
policy_df = pd.DataFrame({"Policy": policy})
policy_summary = policy_df.value_counts().reset_index(name="Transactions")

bucket_stats = []

for bucket in ["BLOCK", "REVIEW", "ALLOW"]:
    idx = (policy == bucket)
    bucket_stats.append({
        "Bucket": bucket,
        "Transactions": int(idx.sum()),
        "Fraud_Rate": y_test[idx].mean() if idx.sum() > 0 else np.nan
    })

policy_stats_df = pd.DataFrame(bucket_stats)

policy_summary.to_csv("policy_distribution.csv", index=False)
policy_stats_df.to_csv("policy_bucket_stats.csv", index=False)

from sklearn.calibration import CalibratedClassifierCV

calibrated_sigmoid = CalibratedClassifierCV(
    estimator=best_model,
    method="sigmoid",
    cv=3
)

calibrated_isotonic = CalibratedClassifierCV(
    estimator=best_model,
    method="isotonic",
    cv=3
)

calibrated_sigmoid.fit(X_train, y_train)
```

```
calibrated_isotonic.fit(X_train, y_train)

probs_raw = best_model.predict_proba(X_test)[:, 1]
probs_sigmoid = calibrated_sigmoid.predict_proba(X_test)[:, 1]
probs_isotonic = calibrated_isotonic.predict_proba(X_test)[:, 1]

calibration_auc = pd.DataFrame({
    "Model": ["Uncalibrated", "Platt-scaled", "Isotonic"],
    "ROC_AUC": [
        roc_auc_score(y_test, probs_raw),
        roc_auc_score(y_test, probs_sigmoid),
        roc_auc_score(y_test, probs_isotonic)
    ]
})

calibration_auc.to_csv("calibration_auc_comparison.csv",  index=False)
```